

Отчёт по лабораторной работе №2

Имитационное моделирование

бахи сиди али темассини

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
2.1	Подготовка рабочего окружения	7
2.2	Выполнение модели SIR	8
2.3	Преобразование модели SIR в литературный стиль	9
2.4	Генерация файлов из литературного кода	9
2.5	Выполнение Jupyter Notebook	10
3	Модель SIR	11
3.1	Инициализация проекта и загрузка пакетов	12
3.2	Определение модели	13
3.3	Параметры модели	14
3.4	Базовое репродуктивное число	15
3.5	Решение системы дифференциальных уравнений	15
3.6	Подготовка результатов	17
3.7	Вывод параметров модели	18
3.8	Динамика основных групп населения	19
3.9	Динамика инфицированных	21
3.10	Логарифмический масштаб	23
3.11	Доли групп населения	24
3.12	Фазовый портрет	27
3.13	Эффективное репродуктивное число	29
3.14	Сводная панель результатов	32
3.15	Сохранение графиков	37
3.16	Оценка производительности	38
3.17	Анализ результатов	38
3.18	Выполнение модели Лотки–Вольтерры	41
3.19	Выполнение литературной версии модели	41
3.20	Генерация файлов литературной программы	42
3.21	Выполнение Jupyter Notebook	42
4	Модель Лотки–Вольтерры	43
4.1	Инициализация проекта	43
4.2	Определение модели	44
4.3	Параметры модели	45

4.4	Начальные условия	46
4.5	Временной интервал	46
4.6	Решение системы	47
4.7	Подготовка результатов	49
4.8	Производные популяций	51
4.9	Параметры модели	52
4.10	Стационарная точка	53
4.11	Динамика популяций	54
4.12	Фазовый портрет	56
4.13	Нулевые изоклины	59
4.14	Скорости изменения популяций	61
4.15	Относительные темпы роста	62
4.16	Спектральный анализ	64
4.17	Сводная панель результатов	68
4.18	Анализ результатов	77
4.19	Дополнительный анализ колебаний	79
5	Выводы	80

Список иллюстраций

2.1	Создание и инициализация рабочего проекта лабораторной работы .	7
2.2	Проверка установленных пакетов рабочего окружения Julia	8
2.3	Результаты выполнения программной модели SIR	8
2.4	Выполнение и структура литературного кода модели SIR	9
2.5	Генерация чистого кода, Quarto-документации и Jupyter Notebook . .	9
2.6	Анализ динамики эффективного репродуктивного числа в Jupyter Notebook	10
3.1	Результаты выполнения модели Лотки–Вольтерры	41
3.2	Результаты выполнения литературной версии модели Лотки–Вольтерры	41
3.3	Генерация файлов из литературной программы модели Лотки–Вольтерры	42
3.4	Визуализация динамики модели Лотки–Вольтерры в Jupyter Notebook	42

Список таблиц

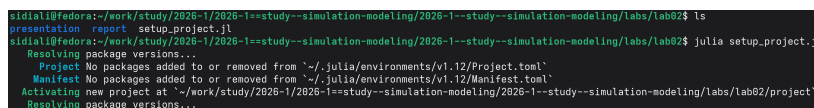
1 Цель работы

Изучить основные математические модели, реализуемые в виде систем обыкновенных дифференциальных уравнений, и освоить их численное моделирование средствами Julia. Исследовать динамику модели SIR и модели Лотки–Вольтерры, выполнить анализ полученных результатов и их визуализацию

2 Выполнение лабораторной работы

2.1 Подготовка рабочего окружения

Для выполнения лабораторной работы был подготовлен отдельный проект lab02/project. Сценарий `setup_project.jl` использован для создания структуры проекта и активации рабочего окружения Julia (рис. 2.1).



```
sidali@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02$ ls
presentation  report  setup_project.jl
sidali@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02$ julia setup_project.jl
Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
Activating new project at `~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project`
Resolving package versions...
```

Рисунок 2.1: Создание и инициализация рабочего проекта лабораторной работы

После создания проекта были установлены необходимые зависимости. Проверка окружения с помощью `Pkg.status()` показала наличие пакетов, используемых для моделирования, обработки данных, построения графиков и подготовки литературного кода (рис. 2.2).

2.3 Преобразование модели SIR в литературный стиль

Программный код модели был преобразован в литературный формат `sir_ode_literary.jl`. В литературном представлении приведены описание модели, группы $S(t)$, $I(t)$ и $R(t)$, используемые параметры и система дифференциальных уравнений. Выполнение литературного сценария подтвердило получение результатов модели SIR (рис. 2.4).

```
julia@fedoras:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project$ julia --project- scripts/sir_ode_literary.jl
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 18.8
γ (скорость выздоровления) = 0.25
R0 = c * β / γ = 2.8
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 999.9, I0 = 18.8, R0 = 0.8
Could not create decoration from factory! Running with no decorations.

Вывод решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_peak = 154.8
Время достижения пика: t_peak = 18.1 дней
Итоговое число переболевших: R(s) = 775.7
Длина переболевших: 77.0%

Точечный анализ:
- Порог коллективного иммунитета: 50.0%
- Теоретический пик при S/N = 1/R0 = 0.5

julia@fedoras:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project$ cat scripts/sir_ode_literary.jl
# Модель SIR
#
# **Цель:** исследовать динамику распространения инфекционного заболевания
# с помощью трайпараметрической модели SIR.
#
# В модели население разделяется на три группы:
#
# - 'S(t)' - восприимчивые;
# - 'I(t)' - инфицированные;
# - 'R(t)' - выздоровевшие.
#
# В трайпараметрической форме используются параметры:
```

Рисунок 2.4: Выполнение и структура литературного кода модели SIR

2.4 Генерация файлов из литературного кода

С помощью сценария `tangle.jl` из файла `sir_ode_literary.jl` были автоматически сформированы чистый программный код, документация Quarto и Jupyter Notebook. Сгенерированные файлы размещены в соответствующих каталогах `scripts`, `markdown` и `notebooks` проекта (рис. 2.5).

```
julia@fedoras:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project$ julia --project- scripts/tangle.jl scripts/sir_ode_literary.jl
Генерация из scripts/sir_ode_literary.jl
[ Info: generating plain script file from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: generating plain script file from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: generating Quarto document from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/markdown/sir_ode_literary.qmd
[ Info: generating Quarto document from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/markdown/sir_ode_literary.qmd
[ Info: generating Jupyter Notebook from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/notebooks/sir_ode_literary.ipynb
[ Info: generating Jupyter Notebook from ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/sir_ode_literary.jl
[ Info: writing result to ~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/notebooks/sir_ode_literary.ipynb
[ Info: All files created.
```

Рисунок 2.5: Генерация чистого кода, Quarto-документации и Jupyter Notebook

2.5 Выполнение Jupyter Notebook

Сгенерированный Jupyter Notebook `sir_ode_literary.ipynb` был выполнен в среде Julia. В ходе выполнения построен график изменения эффективного репродуктивного числа $R_e(t)$ и показан порог эпидемии $R_e = 1$ (рис. 2.6).

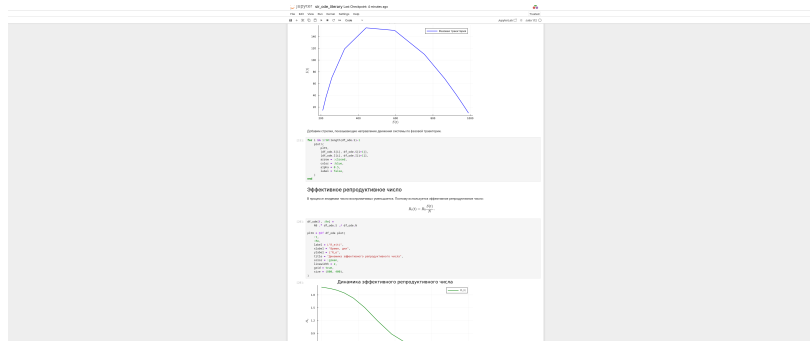


Рисунок 2.6: Анализ динамики эффективного репродуктивного числа в Jupyter Notebook

3 Модель SIR

Цель: исследовать динамику распространения инфекционного заболевания с помощью трёхпараметрической модели SIR.

В модели население разделяется на три группы:

- $S(t)$ — восприимчивые;
- $I(t)$ — инфицированные;
- $R(t)$ — выздоровевшие.

В трёхпараметрической форме используются параметры:

- β — вероятность передачи инфекции при контакте;
- c — среднее число контактов;
- γ — скорость выздоровления.

Система дифференциальных уравнений имеет вид:

$$\frac{dS}{dt} = -\beta c \frac{IS}{N},$$

$$\frac{dI}{dt} = \beta c \frac{IS}{N} - \gamma I,$$

$$\frac{dR}{dt} = \gamma I,$$

где

$$N = S + I + R.$$

3.1 Инициализация проекта и загрузка пакетов

Активируем окружение DrWatson и подключаем библиотеки, необходимые для решения системы ОДУ, обработки данных, визуализации и оценки производительности.

```
using DrWatson
quickactivate("../project", "project")

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
```

Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project`

Имя текущего скрипта используется для создания отдельных каталогов с графиками и результатами вычислений.

```
script_name = splitext(basename(PROGRAM_FILE))[1]

mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/sidiali/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab02/project/data/"
```

3.2 Определение модели

Функция `sir_ode!` задаёт правую часть системы дифференциальных уравнений SIR.

Вектор состояния имеет вид:

$$u = (S, I, R),$$

а вектор параметров:

$$p = (\beta, c, \gamma).$$

```
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p

    N = S + I + R

    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end

    nothing
end
```

`sir_ode!` (generic function with 1 method)

3.3 Параметры модели

Зададим шаг интегрирования, продолжительность моделирования, начальное состояние популяции и параметры распространения инфекции.

```
dt = 0.1  
tmax = 40.0  
tspan = (0.0, tmax)
```

`(0.0, 40.0)`

Начальные условия:

- 990 восприимчивых;
- 10 инфицированных;
- 0 выздоровевших.

```
u0 = [990.0, 10.0, 0.0]
```

3-element Vector{Float64}:

990.0

10.0

0.0

Параметры модели:

- $\beta = 0.05$;
- $c = 10$;
- $\gamma = 0.25$.

```
p = [0.05, 10.0, 0.25]
```

```
3-element Vector{Float64}:
```

```
0.05
```

```
10.0
```

```
0.25
```

3.4 Базовое репродуктивное число

Для трёхпараметрической модели SIR базовое репродуктивное число определяется выражением:

$$R_0 = \frac{c\beta}{\gamma}.$$

Если $R_0 > 1$, число инфицированных на начальном этапе может расти. Если $R_0 < 1$, эпидемический процесс затухает.

```
R0 = (p[2] * p[1]) / p[3]
```

```
2.0
```

3.5 Решение системы дифференциальных уравнений

Создаём объект `ODEProblem`, после чего численно решаем систему на заданном временном интервале.

```
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
```

```
sol_ode = solve(prob_ode, dt = dt)
```

retcode: Success

Interpolation: 3rd order Hermite

t: 15-element Vector{Float64}:

0.0

0.1

0.5331242537191838

1.3919008491040001

2.6102120755226386

4.1811820758323055

6.180123810566044

8.664951930903612

11.691425331813486

15.279370667110317

19.082963490744028

23.423425193442526

28.429493730986167

33.29877917363959

40.0

u: 15-element Vector{Vector{Float64}}:

[990.0, 10.0, 0.0]

[989.4990153200853, 10.247898072276143, 0.25308660763853125]

[987.1852964117763, 11.391111426852982, 1.423592161370628]

[981.8312125227972, 14.026022042698541, 4.142765434504279]

[972.1392310589528, 18.75781556165418, 9.10295337939301]

[954.9905500491583, 27.007702518267603, 18.00174743257403]

[923.0706717929863, 41.92976779832725, 34.9995604086864]

[862.6705810342163, 68.49345419063431, 68.83596477514939]

[754.3022993851439, 109.741861854093, 135.95583876076313]

[595.1014268847732, 150.41363264480688, 254.4849404704199]

```
[442.0141179101298, 154.80694988491848, 403.17893220495165]
[326.8951167493958, 119.06860394684853, 554.0362793037556]
[258.50852421889664, 70.11052194753057, 671.3809538335727]
[227.58338337085735, 37.32857063157991, 735.0880459975626]
[209.8412140201437, 14.473644966455723, 775.6851410134004]
```

3.6 Подготовка результатов

Результат решения преобразуем в DataFrame. Добавим время и общую численность популяции.

```
df_ode = DataFrame(Tables.table(sol_ode'))

rename!(df_ode, ["S", "I", "R"])

df_ode[!, :t] = sol_ode.t

df_ode[!, :N] =
    df_ode.S + df_ode.I + df_ode.R
```

15-element Vector{Float64}:

```
1000.0
1000.0
 999.9999999999999
1000.0
1000.0
 999.9999999999999
1000.0
1000.0
1000.0
```

```
1000.0
1000.0
999.9999999999999
999.9999999999999
999.9999999999999
999.9999999999998
```

3.7 Вывод параметров модели

Выведем используемые параметры, значение базового репродуктивного числа и начальные условия.

```
println("Параметры модели SIR:")
println("β (вероятность заражения) = ", p[1])
println("с (среднее число контактов) = ", p[2])
println("γ (скорость выздоровления) = ", p[3])
println("R0 = с * β / γ = ", round(R0, digits = 3))

println(
  "Средняя продолжительность болезни = ",
  round(1 / p[3], digits = 2),
  " дней",
)

println(
  "Начальные условия: S0 = ",
  u0[1],
  ", I0 = ",
  u0[2],
```

```

    ", R0 = ",
    u0[3],
)

```

Параметры модели SIR:

β (вероятность заражения) = 0.05

c (среднее число контактов) = 10.0

γ (скорость выздоровления) = 0.25

$R0 = c * \beta / \gamma = 2.0$

Средняя продолжительность болезни = 4.0 дней

Начальные условия: $S0 = 990.0$, $I0 = 10.0$, $R0 = 0.0$

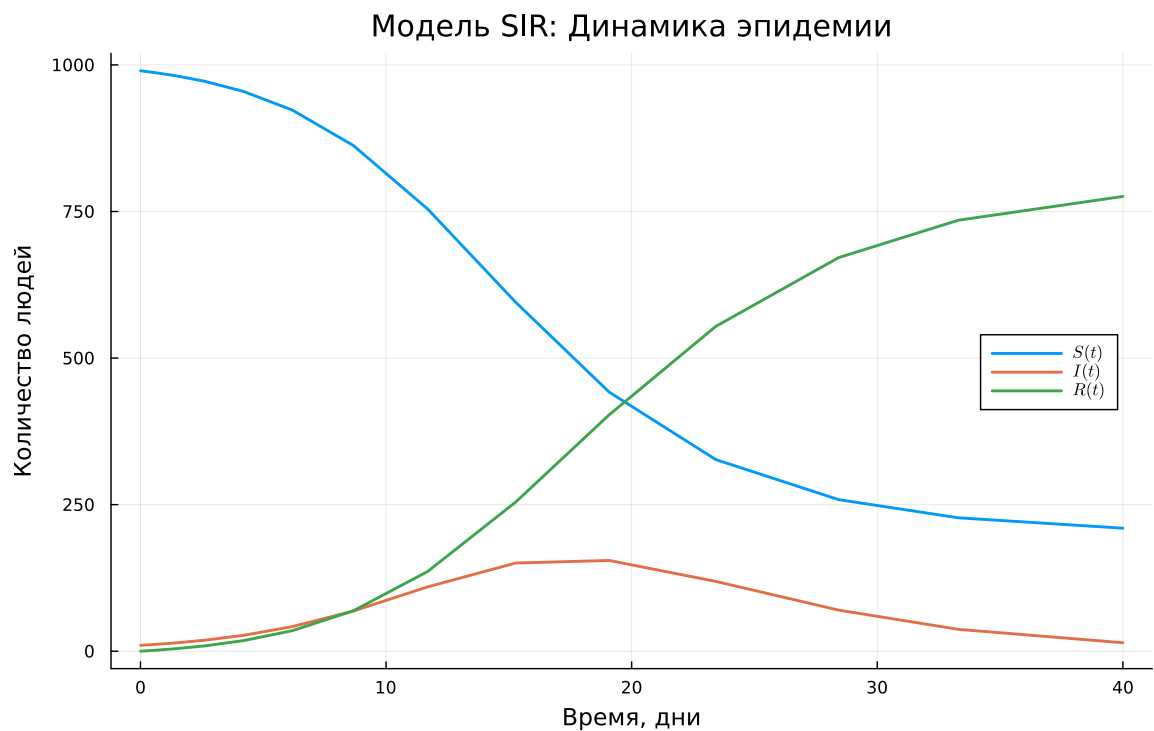
3.8 Динамика основных групп населения

Построим основной график модели, содержащий одновременно траектории $S(t)$, $I(t)$ и $R(t)$.

```

plt1 = @df df_ode plot(
    :t,
    [:S :I :R],
    label = [L"S(t)" L"I(t)" L"R(t)"],
    xlabel = "Время, дни",
    ylabel = "Количество людей",
    title = "Модель SIR: Динамика эпидемии",
    linewidth = 2,
    legend = :right,
    grid = true,
    size = (800, 500),
)

```

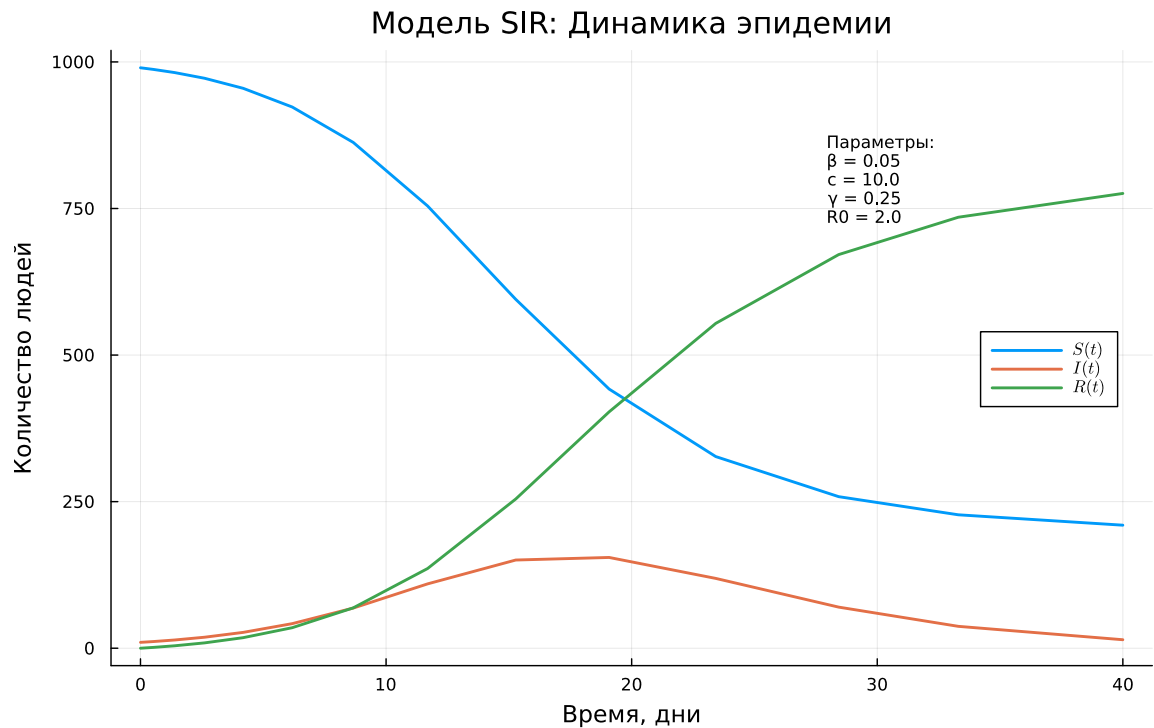


На график добавим основные параметры модели.

```

annotate!(
    plt1,
    maximum(df_ode.t) * 0.7,
    maximum(df_ode.N) * 0.8,
    text(
        "Параметры: \n  $\beta = $(p[1])$  \n  $c = $(p[2])$  \n  $\gamma = $(p[3])$  \n  $R_0 = $(round(R0, digits=2))$ ",
        8,
        :left,
    ),
)

```

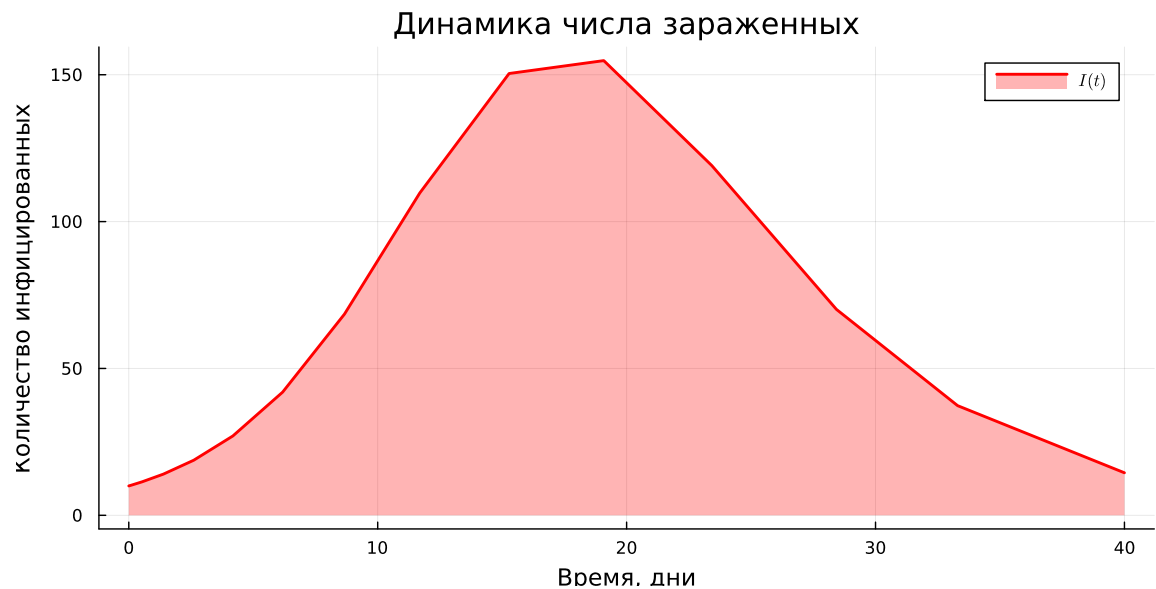


3.9 Динамика инфицированных

Отдельно рассмотрим изменение числа инфицированных $I(t)$.

```
plt2 = @df df_ode plot(
    :t,
    :I,
    label = L"I(t)",
    xlabel = "Время, дни",
    ylabel = "Количество инфицированных",
    title = "Динамика числа зараженных",
    color = :red,
    linewidth = 2,
    fill = (0, 0.3, :red),
    grid = true,
```

```
size = (800, 400),
)
```



3.9.1 Пик эпидемии

Найдём максимальное число инфицированных и момент времени, когда достигается этот максимум.

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]

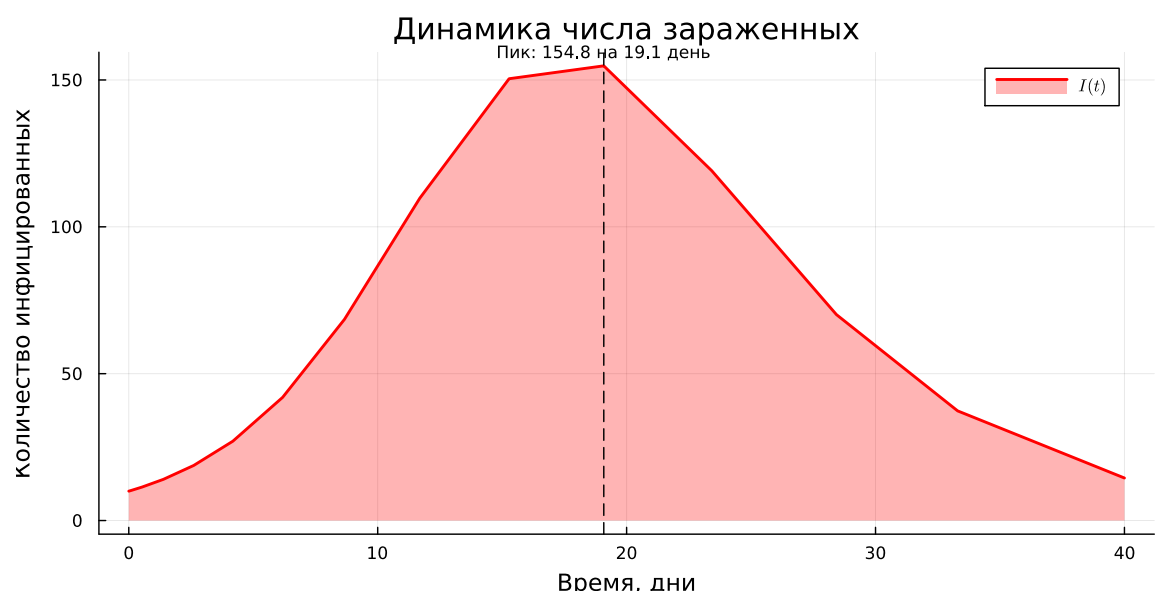
vline!(
    plt2,
    [peak_time],
    color = :black,
    linestyle = :dash,
    label = false,
    linewidth = 1,
```

```

)

annotate!(
    plt2,
    peak_time,
    peak_value * 1.05,
    text(
        "Пик: $(round(peak_value, digits=1)) на $(round(peak_time, digits=1)) день",
        8,
        :top,
    ),
)

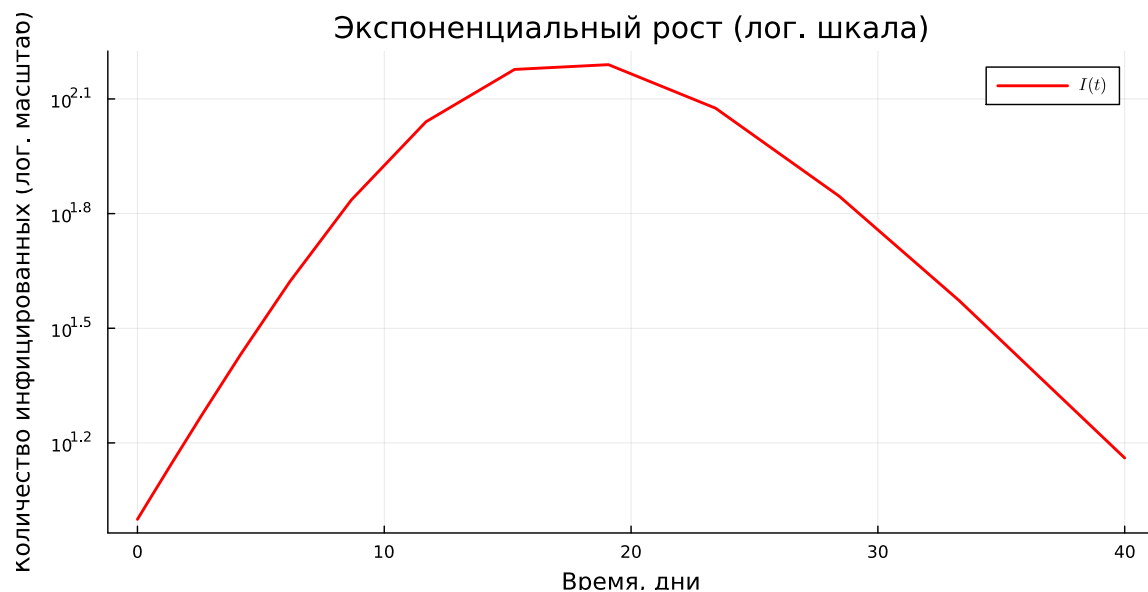
```



3.10 Логарифмический масштаб

Логарифмическая шкала позволяет лучше исследовать начальный экспоненциальный рост числа инфицированных.

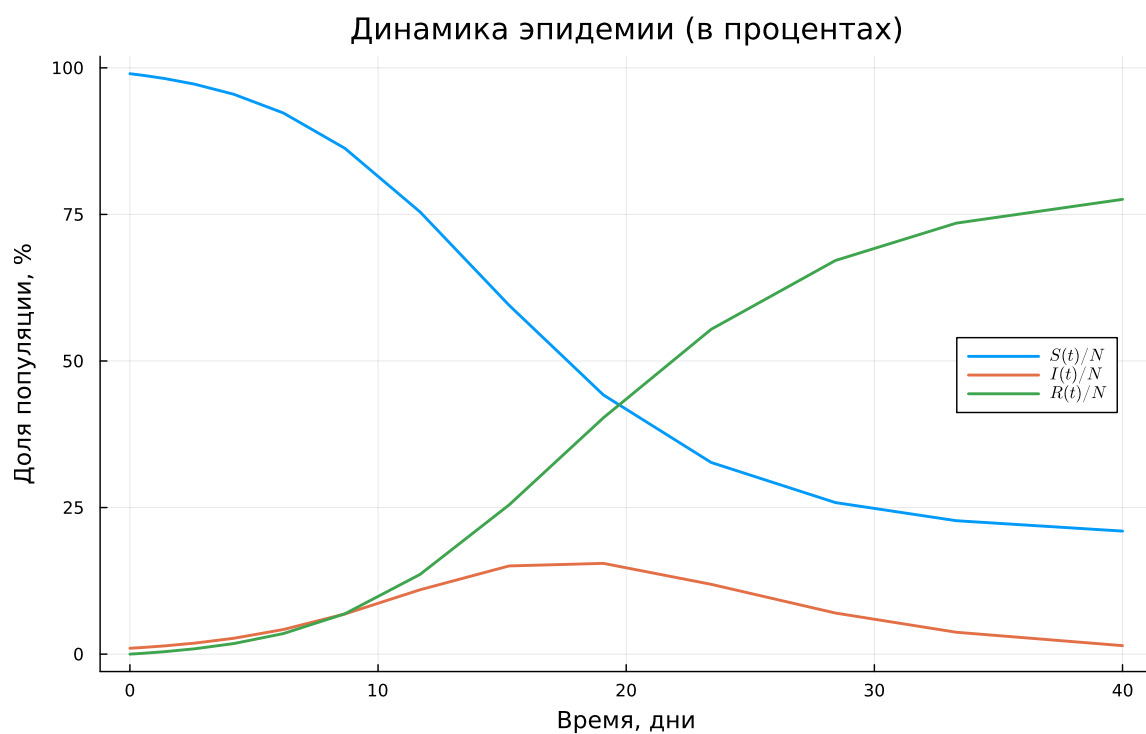
```
plt3 = @df df_ode plot(
    :t,
    :I,
    label = L"I(t)",
    xlabel = "Время, дни",
    ylabel = "Количество инфицированных (лог. масштаб)",
    title = "Экспоненциальный рост (лог. шкала)",
    yscale = :log10,
    color = :red,
    linewidth = 2,
    grid = true,
    size = (800, 400),
)
```



3.11 Доли групп населения

Представим состояния S , I и R не в абсолютных значениях, а в процентах от общей численности населения.

```
plt4 = @df df_ode plot(
    :t,
    [:S :I :R] ./ df_ode.N .* 100,
    label = [L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel = "Время, дни",
    ylabel = "Доля популяции, %",
    title = "Динамика эпидемии (в процентах)",
    linewidth = 2,
    legend = :right,
    grid = true,
    size = (800, 500),
)
```



3.11.1 Порог коллективного иммунитета

Для $R_0 > 1$ теоретический порог коллективного иммунитета:

$$H = 1 - \frac{1}{R_0}.$$

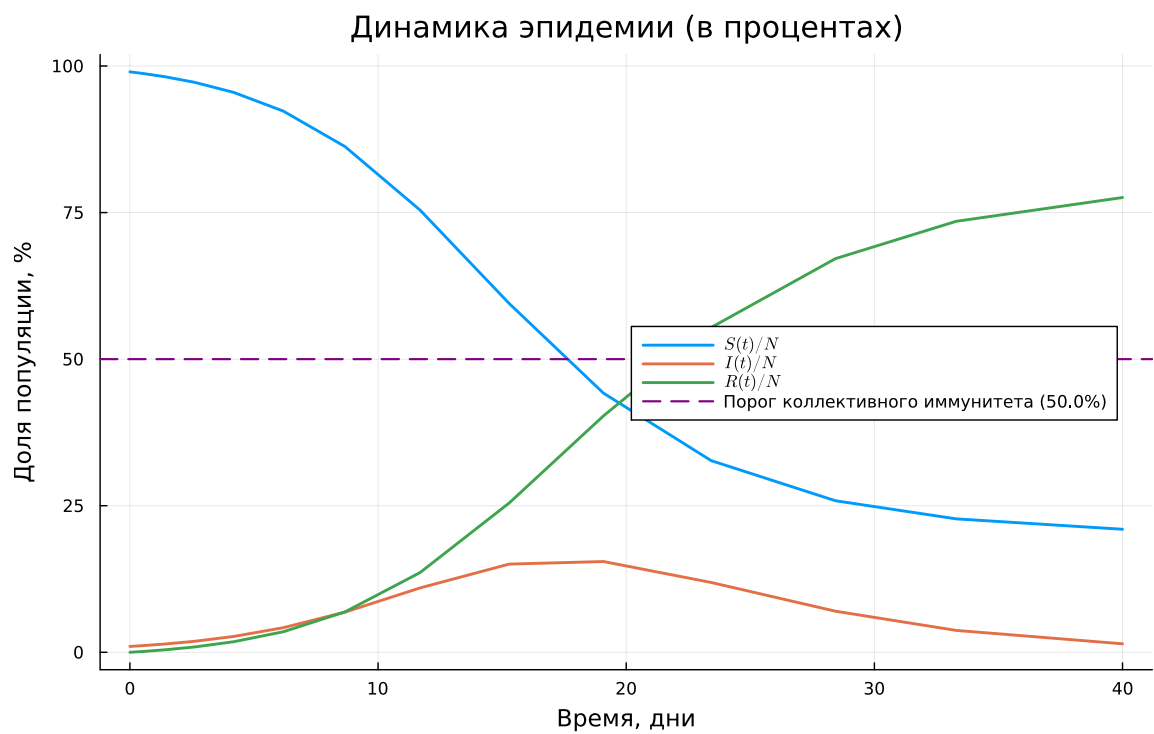
```

if R0 > 1

    herd_immunity_threshold = (1 - 1 / R0) * 100

    hline!(
        plt4,
        [herd_immunity_threshold],
        color = :purple,
        linestyle = :dash,
        label =
            "Порог коллективного иммунитета ($(round(herd_immunity_threshold, digits=1))%)",
        linewidth = 1.5,
    )
end

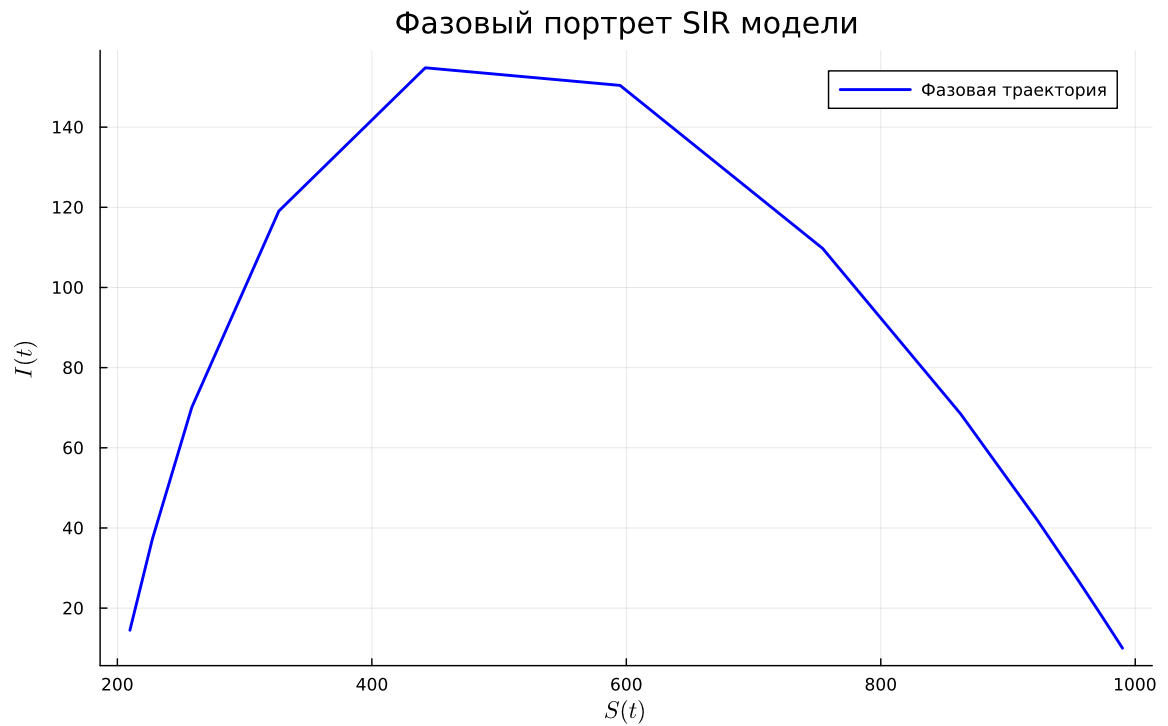
```



3.12 Фазовый портрет

Фазовый портрет показывает взаимосвязь между числом восприимчивых S и числом инфицированных I .

```
plt5 = plot(  
    df_ode.S,  
    df_ode.I,  
    label = "Фазовая траектория",  
    xlabel = L"S(t)",  
    ylabel = L"I(t)",  
    title = "Фазовый портрет SIR модели",  
    color = :blue,  
    linewidth = 2,  
    grid = true,  
    size = (800, 500),  
    legend = :topright,  
)
```



Добавим стрелки, показывающие направление движения системы по фазовой траектории.

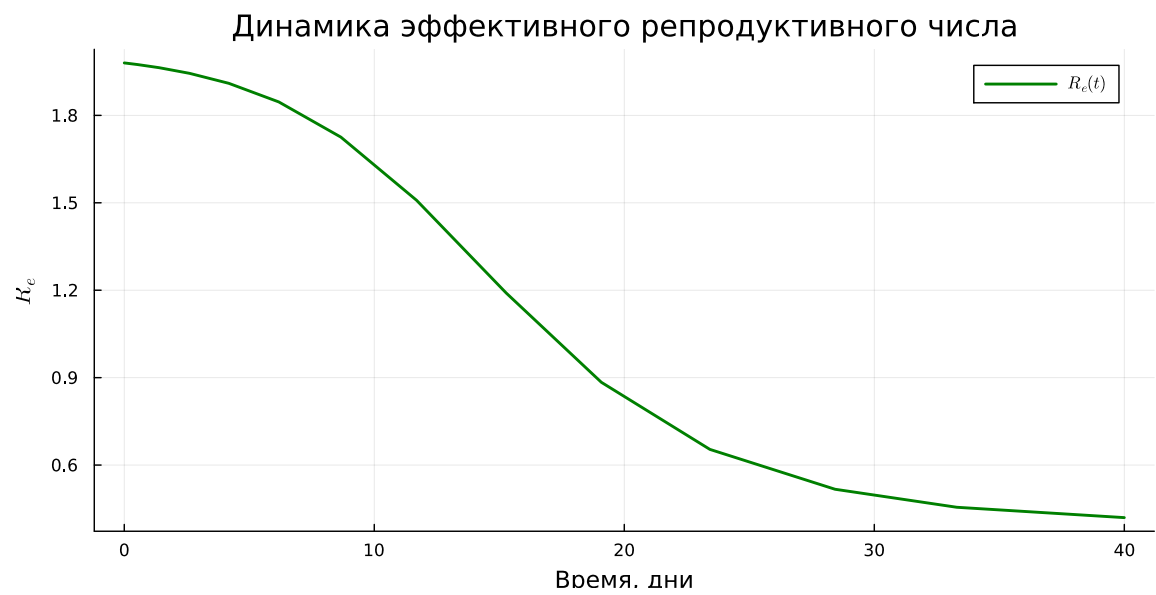
```
for i in 1:50:length(df_ode.S)-1
    plot!(
        plt5,
        [df_ode.S[i], df_ode.S[i+1]],
        [df_ode.I[i], df_ode.I[i+1]],
        arrow = :closed,
        color = :blue,
        alpha = 0.5,
        label = false,
    )
end
```

3.13 Эффективное репродуктивное число

В процессе эпидемии число восприимчивых уменьшается. Поэтому используется эффективное репродуктивное число:

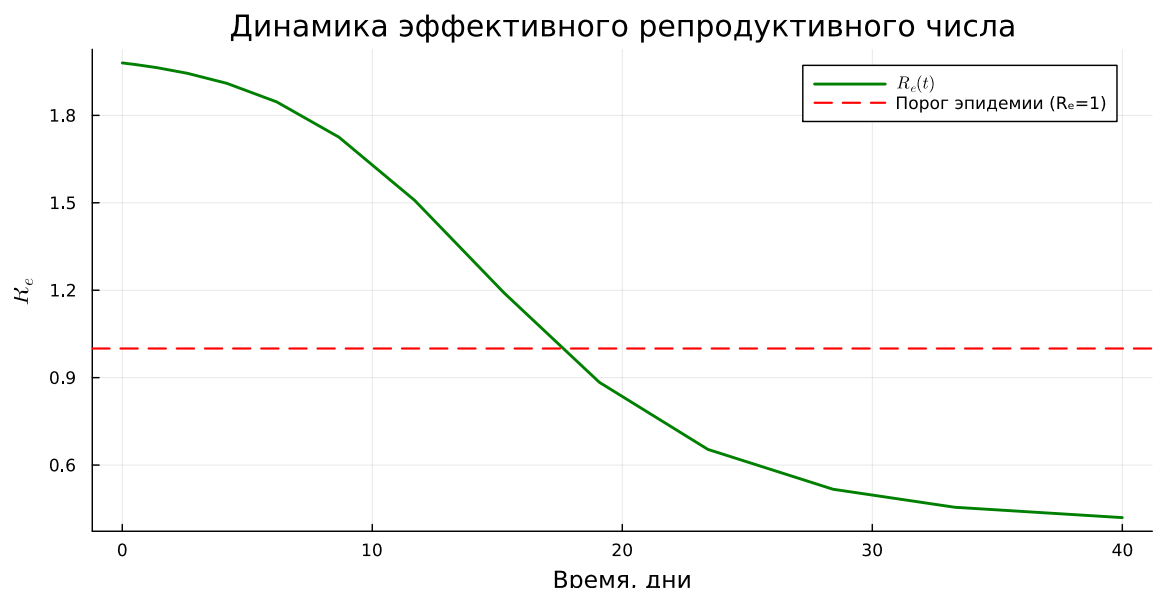
$$R_e(t) = R_0 \frac{S(t)}{N}.$$

```
df_ode[!, :Re] =  
    R0 .* df_ode.S ./ df_ode.N  
  
plt6 = @df df_ode plot(  
    :t,  
    :Re,  
    label = L"R_e(t)",  
    xlabel = "Время, дни",  
    ylabel = L"R_e",  
    title = "Динамика эффективного репродуктивного числа",  
    color = :green,  
    linewidth = 2,  
    grid = true,  
    size = (800, 400),  
)
```



Значение $R_e = 1$ является пороговым.

```
hline(
    plt6,
    [1.0],
    color = :red,
    linestyle = :dash,
    label = "Порог эпидемии ( $R_e=1$ )",
    linewidth = 1.5,
)
```



Найдём момент, когда эффективное репродуктивное число впервые становится меньше единицы.

```
cross_idx = findfirst(x → x < 1, df_ode.Re)

if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]

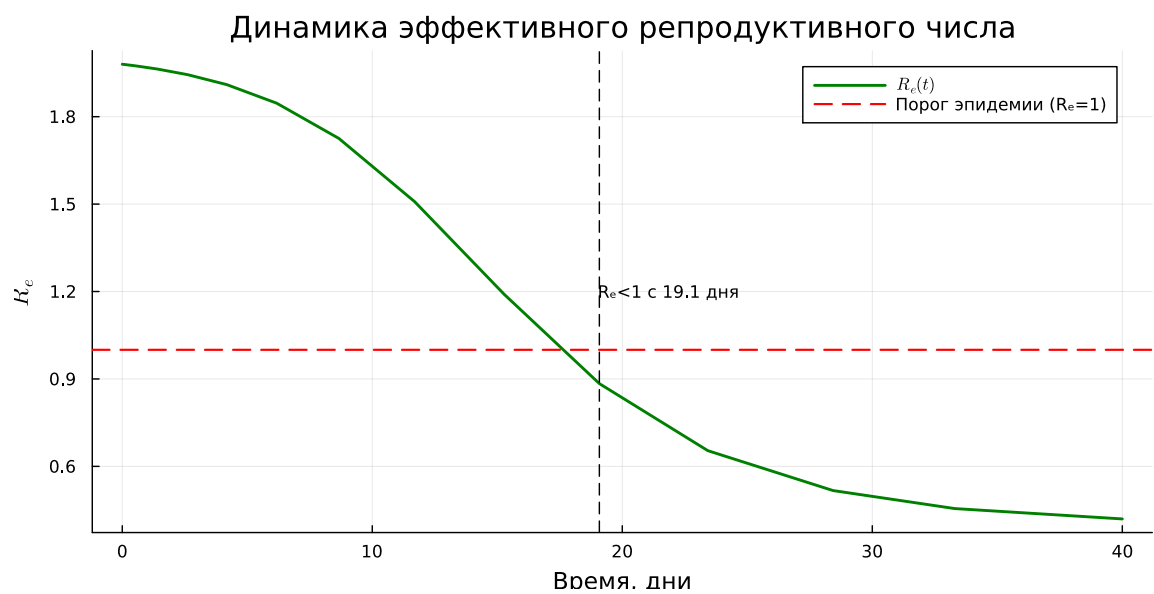
    vline!(
        plt6,
        [cross_time],
        color = :black,
        linestyle = :dash,
        label = false,
        linewidth = 1,
    )

    annotate!(
```

```

plt6,
cross_time,
1.2,
text(
    "Re<1 с $(round(cross_time, digits=1)) дня",
    8,
:left,
),
)
end

```



3.14 Сводная панель результатов

Для удобного сравнения основных характеристик модели разместим несколько графиков на одной панели.

```

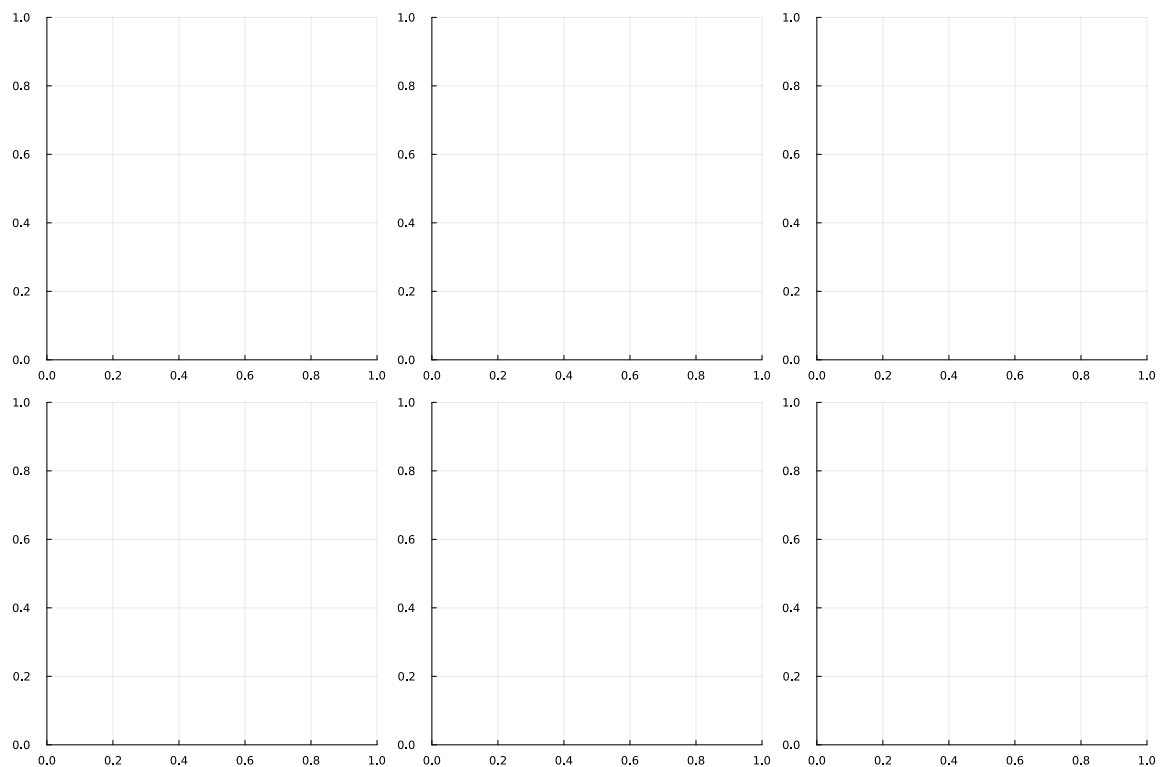
plt7 = plot(
    layout = (2, 3),

```

```

size = (1200, 800),
)

```



3.14.1 Верхний ряд

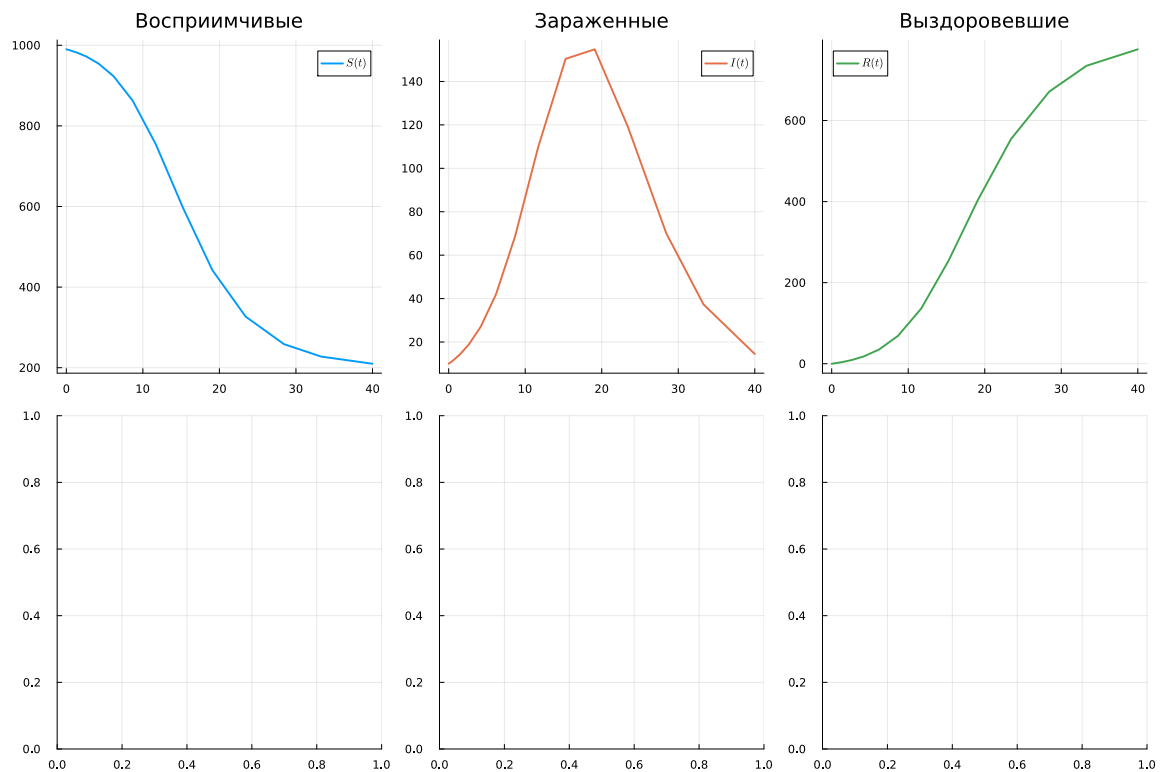
```

plot!(
    plt7[1],
    df_ode.t,
    df_ode.S,
    label = L"S(t)",
    color = 1,
    linewidth = 2,
    title = "Восприимчивые",
)

```

```
plot!(
    plt7[2],
    df_ode.t,
    df_ode.I,
    label = L"I(t)",
    color = 2,
    linewidth = 2,
    title = "Зараженные",
)

plot!(
    plt7[3],
    df_ode.t,
    df_ode.R,
    label = L"R(t)",
    color = 3,
    linewidth = 2,
    title = "Выздоровевшие",
)
```



3.14.2 Нижний ряд

```
plot!(
    plt7[4],
    df_ode.t,
    df_ode.I,
    label = L"I(t)",
    color = 2,
    linewidth = 2,
    yscale = :log10,
    title = "Лог. масштаб",
)

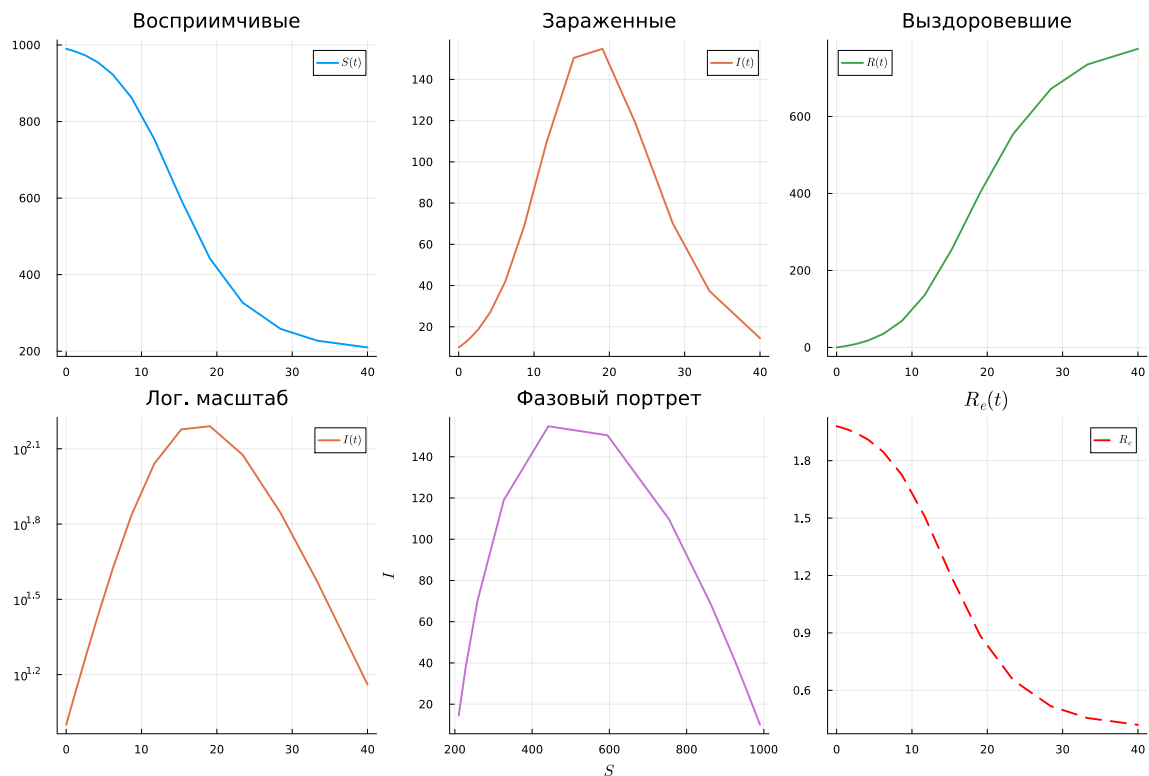
plot!(
```

```

plt7[5],
df_ode.S,
df_ode.I,
label = false,
color = 4,
linewidth = 2,
title = "Фазовый портрет",
xlabel = L"S",
ylabel = L"I",
)

plot!(
    plt7[6],
    df_ode.t,
    df_ode.Re,
    label = L"R_e",
    color = :green,
    linewidth = 2,
    title = L"R_e(t)",
    hline = [1.0],
    linestyle = :dash,
    linecolor = :red,
)

```



3.15 Сохранение графиков

Сохраним все построенные графики в каталог `plots/sir_ode/`.

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"/home/sidiali/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--
study--simulation-modeling/labs/lab02/project/plots/sir_panel.png"
```

3.16 Оценка производительности

Используем BenchmarkTools для измерения производительности численного решения системы.

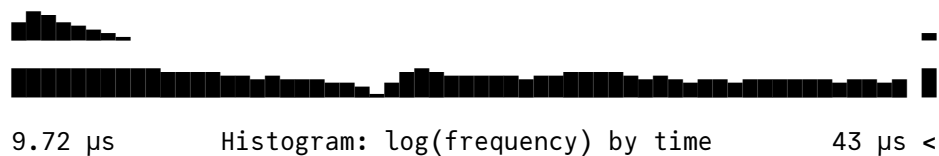
```
println("\nБенчмарк решения:")

@benchmark solve(prob_ode, dt = δt)
```

Бенчмарк решения:

BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.

Range (min ... max):	9.722 μ s ... 8.483 ms	GC (min ... max):	0.00% ... 99.10%
Time (median):	10.819 μ s	GC (median):	0.00%
Time (mean $\pm$ σ):	14.651 μ s $\pm$ 141.359 μ s	GC (mean $\pm$ σ):	16.53% $\pm$ 1.72%



Memory estimate: 21.85 KiB, allocs estimate: 368.

3.17 Анализ результатов

Выведем основные количественные характеристики эпидемического процесса.

```
println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")

println(
    "Общая численность популяции (контроль): N = ",
```

```

        round(df_ode.N[1], digits = 1),
    )

println(
    "Пиковое число зараженных: I_max = ",
    round(peak_value, digits = 1),
)

println(
    "Время достижения пика: t_peak = ",
    round(peak_time, digits = 1),
    " дней",
)

println(
    "Итоговое число переболевших: R( $\infty$ ) = ",
    round(df_ode.R[end], digits = 1),
)

println(
    "Доля переболевших: ",
    round(
        df_ode.R[end] / df_ode.N[1] * 100,
        digits = 1,
    ),
    "%",
)

```

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===

Общая численность популяции (контроль): $N = 1000.0$

Пиковое число зараженных: $I_{\max} = 154.8$

Время достижения пика: $t_{\text{peak}} = 19.1$ дней

Итоговое число переболевших: $R(\infty) = 775.7$

Доля переболевших: 77.6%

Для $R_0 > 1$ дополнительно выведем теоретические характеристики модели.

```
if R0 > 1
    println("\nТеоретический анализ:")

    println(
        " - Порог коллективного иммунитета: ",
        round((1 - 1 / R0) * 100, digits = 1),
        "%",
    )

    println(
        " - Теоретический пик при  $S/N = 1/R_0 =$ ",
        round(1 / R0, digits = 3),
    )
end
```

Теоретический анализ:

- Порог коллективного иммунитета: 50.0%
- Теоретический пик при $S/N = 1/R_0 = 0.5$

3.18 Выполнение модели Лотки–Вольтерры

Модель Лотки–Вольтерры была выполнена командой `julia --project=. scripts/lv_ode.jl`. Использовались параметры $\alpha = 0.1$, $\beta = 0.02$, $\delta = 0.01$ и $\gamma = 0.3$ при начальных условиях: 40 жертв и 9 хищников. Расчёт показал стационарную точку $(x^*, y^*) = (30, 5)$ и периодический характер изменения популяций (рис. 3.1).

```
julia@fedora:~/work/study/2020-1/2020-1--study--simulation-modeling/2020-1--study--simulation-modeling/labs/lab02/project$ julia --project=. scripts/lv_ode.jl
[Warning: Verbosity toggle: dense_output_save!
Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two behaviors.
]
=====
Модель Лотки-Вольтерры (хищник-жертва)
=====
Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конкуренции) = 0.01
γ (смертность хищников) = 0.3
Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0
Стационарные точки (положения равновесия):
x* = y/β = 30.0
y* = α/γ = 5.0
Дименирующая частота колебаний жертв: 0.2429 Гц
Период колебаний жертв: 4.08 единиц времени
=====
Анализ результатов
=====
Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2
Анализ колебаний:
Период пик жертв: время = 33.7, значение = 46.89
```

Рисунок 3.1: Результаты выполнения модели Лотки–Вольтерры

3.19 Выполнение литературной версии модели

Литературная версия `lv_ode_literary.jl` была выполнена в активном окружении проекта. Получены параметры модели, стационарные точки и основные статистические характеристики популяций. Для жертв значения изменялись от 17.75 до 46.89, а для хищников — от 1.9 до 10.41 (рис. 3.2).

```
Моделирование завершено успешно
julia@fedora:~/work/study/2020-1/2020-1--study--simulation-modeling/2020-1--study--simulation-modeling/labs/lab02/project$ nano scripts/lv_ode_literary.jl
julia@fedora:~/work/study/2020-1/2020-1--study--simulation-modeling/2020-1--study--simulation-modeling/labs/lab02/project$ julia --project=. scripts/lv_ode_literary.jl
[Warning: Verbosity toggle: dense_output_save!
Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two behaviors.
]
=====
Модель Лотки-Вольтерры (хищник-жертва)
=====
Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конкуренции) = 0.01
γ (смертность хищников) = 0.3
Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0
Стационарные точки (положения равновесия):
x* = y/β = 30.0
y* = α/γ = 5.0
Дименирующая частота колебаний жертв: 0.2429 Гц
Период колебаний жертв: 4.08 единиц времени
=====
Анализ результатов
=====
Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2
julia@fedora:~/work/study/2020-1/2020-1--study--simulation-modeling/2020-1--study--simulation-modeling/labs/lab02/project$
```

Рисунок 3.2: Результаты выполнения литературной версии модели Лотки–Вольтерры

3.20 Генерация файлов литературной программы

С помощью сценария `tangle.jl` из файла `lv_ode_literary.jl` были сформированы чистый программный файл, документ Quarto и Jupyter Notebook. Результаты сохранены в соответствующих каталогах `scripts`, `markdown` и `notebooks` (рис. 3.3).

```
Julia 1.9.0-rc1: ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project
julia -p scripts/tangle.jl scripts/lv_ode_literary.jl
[info] generating plain script file from ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] writing result to ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] generating plain script file from ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] writing result to ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] generating quarto document from ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] writing result to ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/markdown/lv_ode_literary.jl
[info] generating jupyter notebook from ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode_literary.jl
[info] writing result to ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/lv_ode_literary.jl.ipynb
[info] Notebook: /home/sidali/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/lv_ode_literary.jl.ipynb
[info] Все файлы созданы.
Julia 1.9.0-rc1: ~/work/study/2026-1/study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project
```

Рисунок 3.3: Генерация файлов из литературной программы модели Лотки–Вольтерры

3.21 Выполнение Jupyter Notebook

Сгенерированный Jupyter Notebook `lv_ode_literary.ipynb` был выполнен для визуального анализа модели. Получены графики динамики популяций жертв и хищников, фазовый портрет и скорости изменения популяций (рис. 3.4).

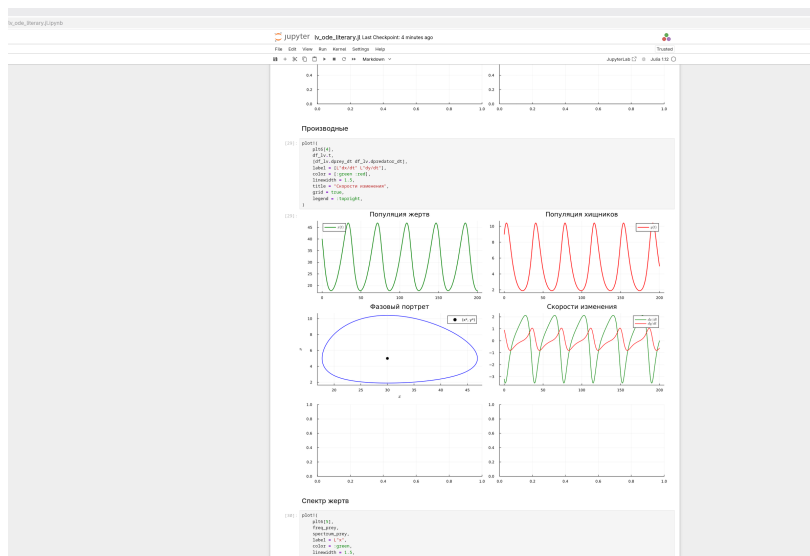


Рисунок 3.4: Визуализация динамики модели Лотки–Вольтерры в Jupyter Notebook

4 Модель Лотки–Вольтерры

Цель: исследовать взаимодействие двух популяций — жертв и хищников — с помощью классической модели Лотки–Вольтерры.

В модели используются две переменные состояния:

- $x(t)$ — популяция жертв;
- $y(t)$ — популяция хищников.

Система дифференциальных уравнений имеет вид:

$$\frac{dx}{dt} = \alpha x - \beta xy,$$

$$\frac{dy}{dt} = \delta xy - \gamma y.$$

Здесь:

- α — естественный прирост жертв;
- β — интенсивность поедания жертв хищниками;
- δ — коэффициент прироста хищников за счёт питания;
- γ — естественная смертность хищников.

4.1 Инициализация проекта

Активируем окружение DrWatson и подключаем библиотеки, необходимые для численного решения, анализа данных, визуализации, статистики и спектрального

анализа.

```
using DrWatson
quickactivate("../project", "project")

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
```

Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project`

Имя текущего скрипта используется для создания отдельных каталогов с результатами и графиками.

```
script_name = splitext(basename(PROGRAM_FILE))[1]

mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

"/home/sidiali/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/data/"

4.2 Определение модели

Функция `lotka_volterra!` задаёт правую часть системы дифференциальных уравнений.

Вектор состояния:

$$u = (x, y),$$

а вектор параметров:

$$p = (\alpha, \beta, \delta, \gamma).$$

```
function lotka_volterra!(du, u, p, t)
    x, y = u
    α, β, δ, γ = p

    @inbounds begin
        du[1] = α * x - β * x * y
        du[2] = δ * x * y - γ * y
    end

    nothing
end
```

lotka_volterra! (generic function with 1 method)

4.3 Параметры модели

Используем классический набор параметров для системы «хищник–жертва».

```
p_lv = [
    0.1, # α: скорость размножения жертв
    0.02, # β: скорость поедания жертв хищниками
    0.01, # δ: коэффициент конверсии пищи в хищников
```

```
0.3, # γ: смертность хищников  
]
```

4-element Vector{Float64}:

```
0.1  
0.02  
0.01  
0.3
```

4.4 Начальные условия

В начале моделирования имеется 40 жертв и 9 хищников.

```
u0_lv = [40.0, 9.0]
```

2-element Vector{Float64}:

```
40.0  
9.0
```

4.5 Временной интервал

Моделирование проводится на интервале от 0 до 200 единиц модельного времени.

```
tspan_lv = (0.0, 200.0)  
dt_lv = 0.01
```

```
0.01
```

4.6 Решение системы

Создадим задачу `ODEProblem` и решим её методом `Tsit5`. Для повышения точности зададим относительную и абсолютную погрешности.

```
prob_lv = ODEProblem(  
    lotka_volterra!,  
    u0_lv,  
    tspan_lv,  
    p_lv,  
)
```

```
sol_lv = solve(  
    prob_lv,  
    Tsit5(),  
    dt = dt_lv,  
    reltol = 1e-8,  
    abstol = 1e-10,  
    saveat = 0.1,  
    dense = true,  
)
```

```
└ Warning: Verbosity toggle: dense_output_saveat  
  | Dense output is incompatible with saveat. Please use the SavingCallback from the Callbacks package.  
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/niPe0/src/solve.jl:410
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 2001-element Vector{Float64}:

0.0

```

0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
:
198.9
199.0
199.1
199.2
199.3
199.4
199.5
199.6
199.7
199.8
199.9
200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890384, 9.088990237207637]
 [39.35113789503168, 9.175882726306998]

```

[39.02053587808482, 9.260562174341327]
 [38.68624820281499, 9.342916311942014]
 [38.34860018759226, 9.422836265415745]
 [38.007919320045765, 9.5002169187062]
 [37.664534369328166, 9.574957250330058]
 [37.318774466548234, 9.646960666854817]
 [36.9709682176784, 9.716135308317774]
 [36.62144287329975, 9.782394321389946]
 [36.270523365122514, 9.845656154267312]
 [35.918531633114334, 9.905844751305578]
 :
 [17.889675634214093, 5.712401894068274]
 [17.86543624231871, 5.643570483722274]
 [17.843673453877326, 5.575440219891977]
 [17.824353144929287, 5.508019318506822]
 [17.807442037033727, 5.441315154551622]
 [17.792907686320646, 5.375334300040277]
 [17.780718473179295, 5.310082577777409]
 [17.77084358854019, 5.245565084811742]
 [17.763253013148603, 5.18178624548093]
 [17.757917517459237, 5.1187498117210595]
 [17.754808657966535, 5.056458873592443]
 [17.75389874795968, 4.994915937053457]

4.7 Подготовка результатов

Преобразуем решение системы в таблицу DataFrame. Для каждого момента времени сохраняются численности жертв и хищников.

```
df_lv = DataFrame()

df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u]
df_lv[!, :predator] = [u[2] for u in sol_lv.u]
```

2001-element Vector{Float64}:

```
9.0
9.088990237207637
9.175882726306998
9.260562174341327
9.342916311942014
9.422836265415745
9.5002169187062
9.574957250330058
9.646960666854817
9.716135308317774
9.782394321389946
9.845656154267312
9.905844751305578
:
5.712401894068274
5.643570483722274
5.575440219891977
5.508019318506822
5.441315154551622
5.375334300040277
5.310082577777409
5.245565084811742
```

```
5.18178624548093
5.1187498117210595
5.056458873592443
4.994915937053457
```

4.8 Производные популяций

Для дальнейшего анализа вычислим скорости изменения численности жертв и хищников непосредственно по формулам модели.

```
df_lv[!, :dprey_dt] =
    p_lv[1] .* df_lv.prey .-
    p_lv[2] .* df_lv.prey .* df_lv.predator

df_lv[!, :dpredator_dt] =
    p_lv[3] .* df_lv.prey .* df_lv.predator .-
    p_lv[4] .* df_lv.predator
```

2001-element Vector{Float64}:

```
0.90000000000000004
0.8796081183812872
0.8580494468233599
0.8353523334488111
0.8115489002365717
0.7866749261310111
0.7607697060793308
0.7338758893000268
0.7060392940837215
0.677308704329461
0.6477356516317538
```

```
0.6173741696029547
0.5862805551332171
:
-0.691790398448966
-0.6848226585569617
-0.6777687195139287
-0.6706369809304632
-0.6634355041663743
-0.6561720201747865
-0.6488539394852796
-0.6414883588865523
-0.6340820722589678
-0.6266415740257921
-0.6191730642026245
-0.6116824631058648
```

4.9 Параметры модели

Выведем основные параметры и начальные условия.

```
println("="^60)
println("Модель Лотки-Вольтерры (хищник-жертва)")
println("="^60)

println("\nПараметры модели:")
println("α (скорость размножения жертв) = ", p_lv[1])
println("β (скорость поедания жертв) = ", p_lv[2])
println("δ (коэффициент конверсии) = ", p_lv[3])
println("γ (смертность хищников) = ", p_lv[4])
```

```
println("\nНачальные условия:")
println("Жертвы (x0) = ", u0_lv[1])
println("Хищники (y0) = ", u0_lv[2])
```

```
=====
Модель Лотки-Вольтерры (хищник-жертва)
=====
```

Параметры модели:

α (скорость размножения жертв) = 0.1

β (скорость поедания жертв) = 0.02

δ (коэффициент конверсии) = 0.01

γ (смертность хищников) = 0.3

Начальные условия:

Жертвы (x0) = 40.0

Хищники (y0) = 9.0

4.10 Стационарная точка

Положительное положение равновесия системы определяется как:

$$x^* = \frac{\gamma}{\delta},$$

$$y^* = \frac{\alpha}{\beta}.$$

```

x_star = p_lv[4] / p_lv[3]
y_star = p_lv[1] / p_lv[2]

println("\nСтационарные точки (положения равновесия):")
println("x* =  $\gamma/\delta$  = ", round(x_star, digits = 3))
println("y* =  $\alpha/\beta$  = ", round(y_star, digits = 3))

```

Стационарные точки (положения равновесия):

$$x^* = \gamma/\delta = 30.0$$

$$y^* = \alpha/\beta = 5.0$$

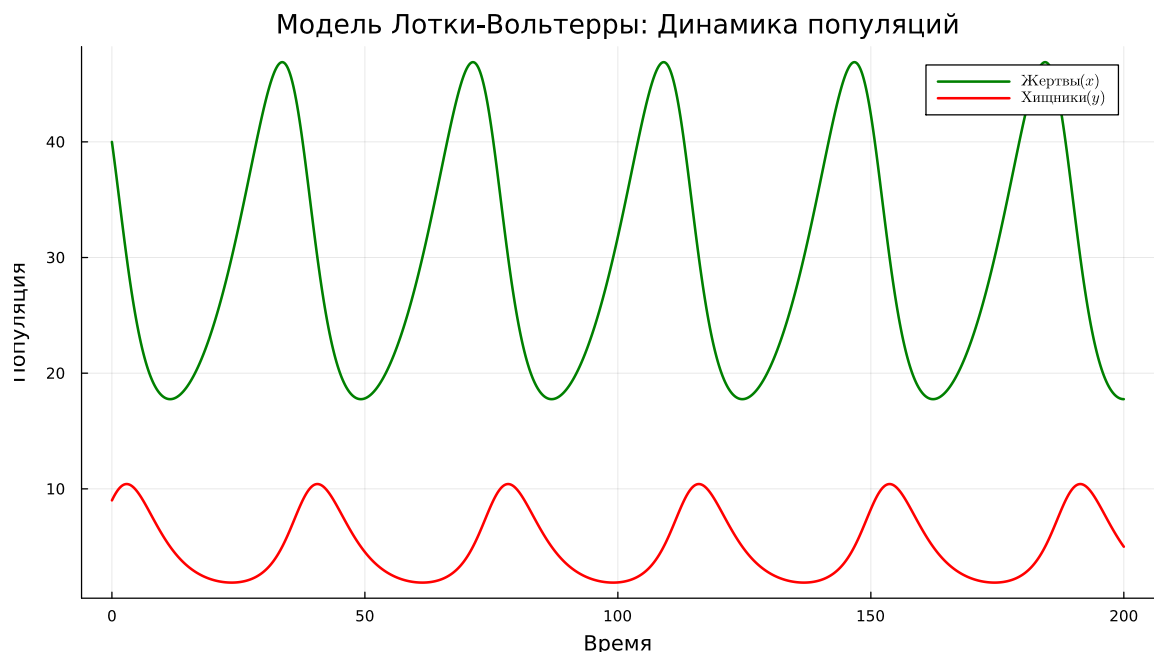
4.11 Динамика популяций

Построим временные зависимости численности жертв и хищников.

```

plt1 = plot(
    df_lv.t,
    [df_lv.prey df_lv.predator],
    label = [L"Жертвы (x)" L"Хищники (y)"],
    xlabel = "Время",
    ylabel = "Популяция",
    title = "Модель Лотки-Вольтерры: Динамика популяций",
    linewidth = 2,
    legend = :topright,
    grid = true,
    size = (900, 500),
    color = [:green :red],
)

```

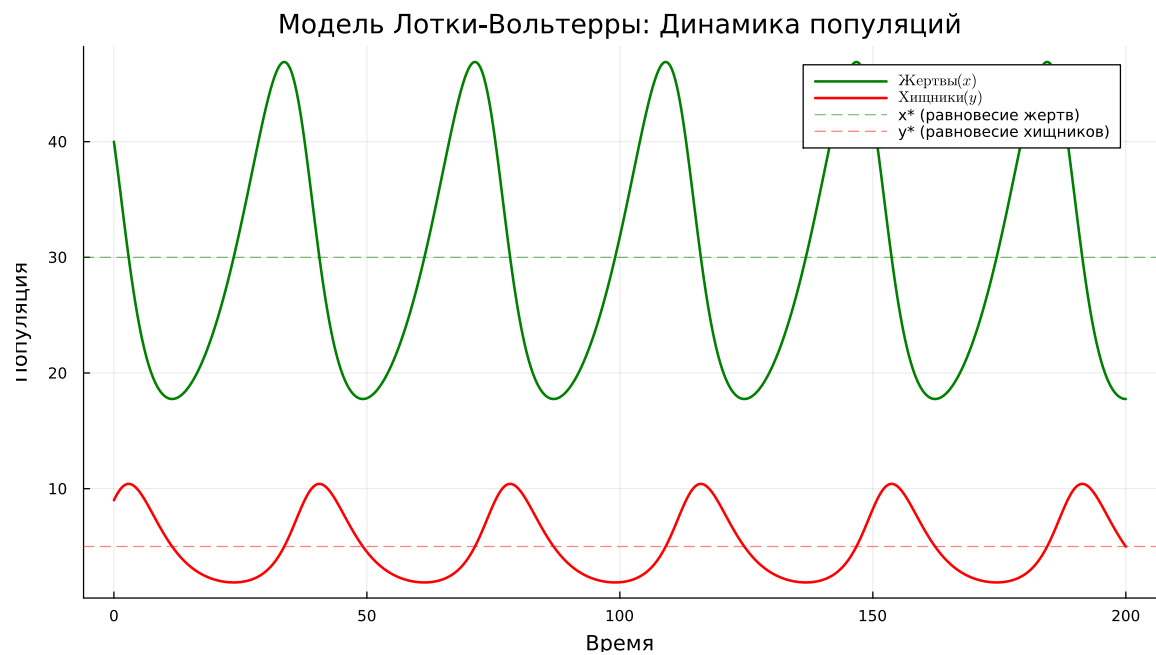


Добавим на график стационарные уровни обеих популяций.

```
hline!(
    plt1,
    [x_star],
    color = :green,
    linestyle = :dash,
    alpha = 0.5,
    label = "x* (равновесие жертв)",
)
```

```
hline!(
    plt1,
    [y_star],
    color = :red,
    linestyle = :dash,
    alpha = 0.5,
```

```
label = "y* (равновесие хищников)",
)
```



4.12 Фазовый портрет

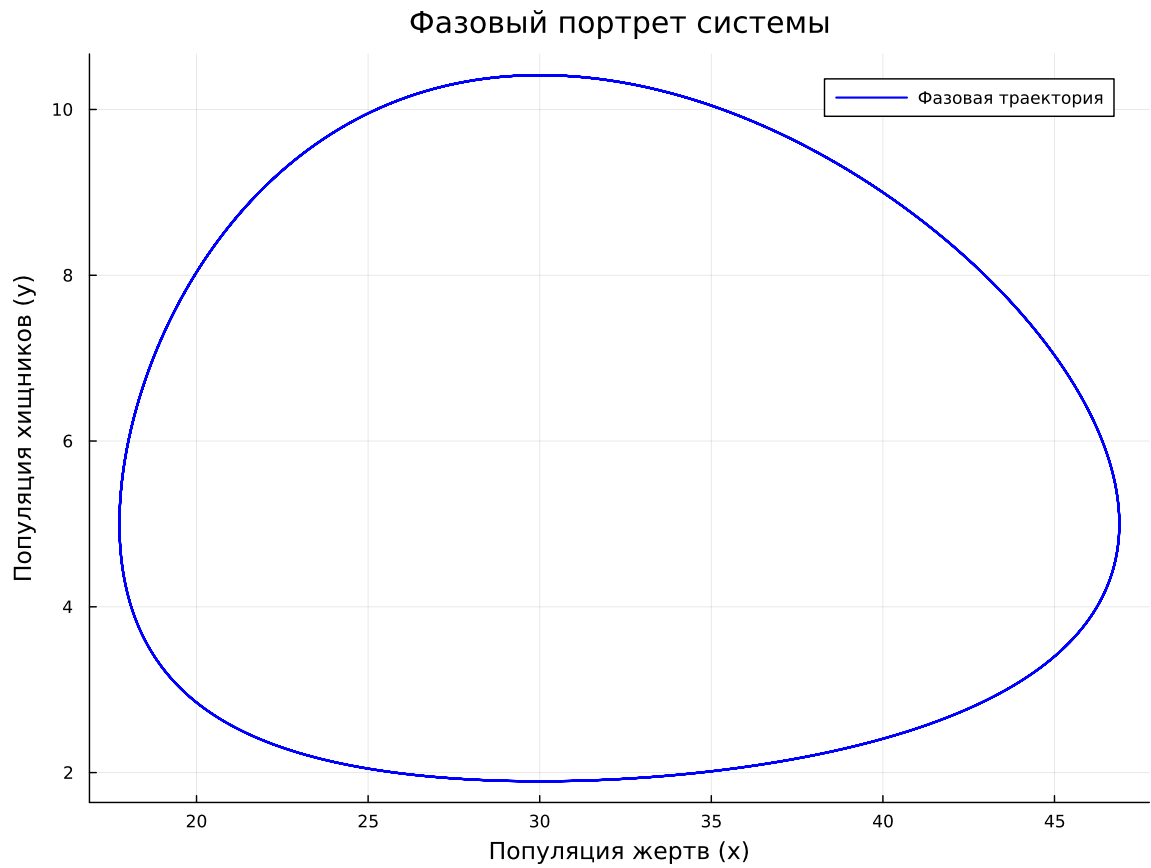
Фазовая траектория показывает взаимосвязь между числом жертв и числом хищников.

```
plt2 = plot(
    df_lv.prey,
    df_lv.predator,
    label = "Фазовая траектория",
    xlabel = "Популяция жертв (x)",
    ylabel = "Популяция хищников (y)",
    title = "Фазовый портрет системы",
    color = :blue,
    linewidth = 1.5,
```

```

grid = true,
size = (800, 600),
legend = :topright,
)

```



4.12.1 Направление фазовой траектории

Добавим стрелки, показывающие направление движения системы по фазовому портрету.

```

step = 50

for i in 1:step:length(df_lv.prey)-step
    plot!(

```

```

        plt2,
        [df_lv.prey[i], df_lv.prey[i+step]],
        [df_lv.predator[i], df_lv.predator[i+step]],
        arrow = :closed,
        color = :blue,
        alpha = 0.3,
        label = false,
    )
end

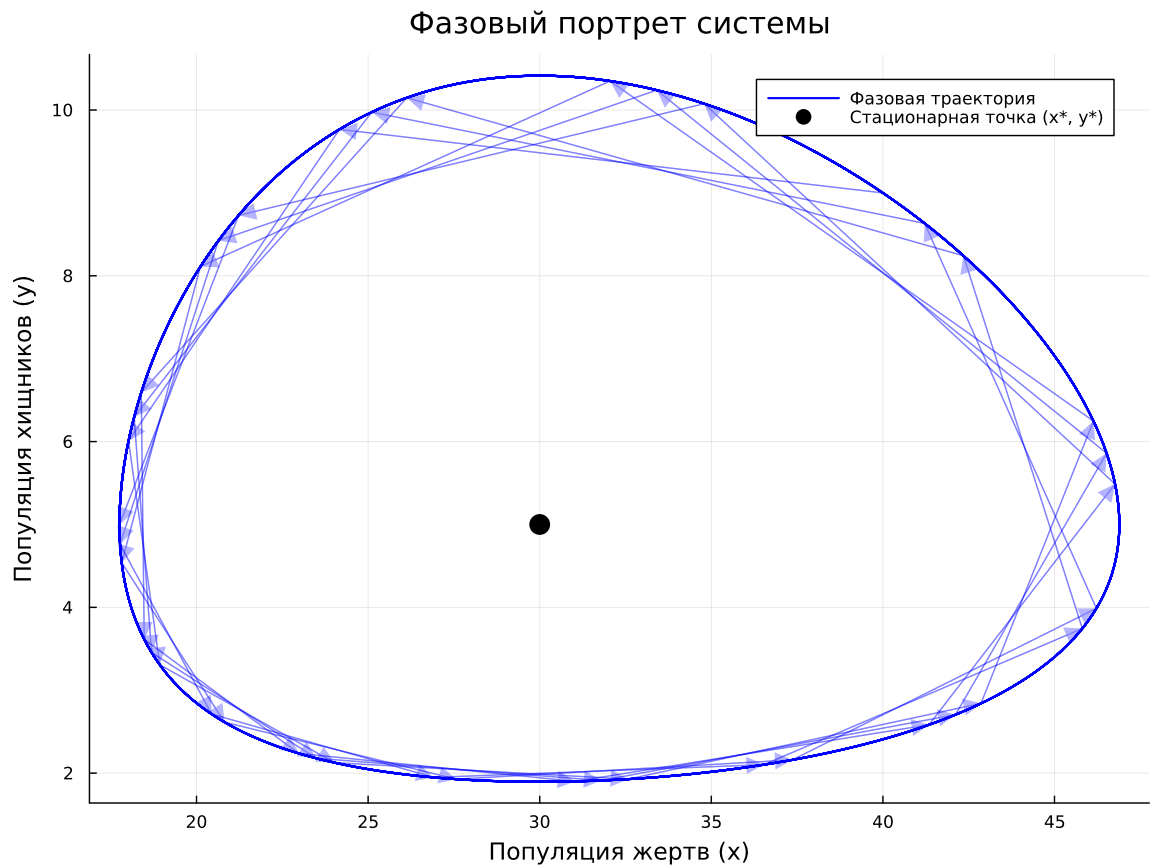
```

Отметим стационарную точку.

```

scatter!(
    plt2,
    [x_star],
    [y_star],
    color = :black,
    markersize = 8,
    label = "Стационарная точка (x*, y*)",
)

```



4.13 Нулевые изоклины

Нулевые изоклины показывают положения, в которых скорость изменения соответствующей популяции равна нулю.

```
x_range =
    LinRange(0, maximum(df_lv.prey) * 1.1, 100)

y_nullcline =
    p_lv[1] ./ (p_lv[2] .* x_range)

plot!(
    plt2,
```

```

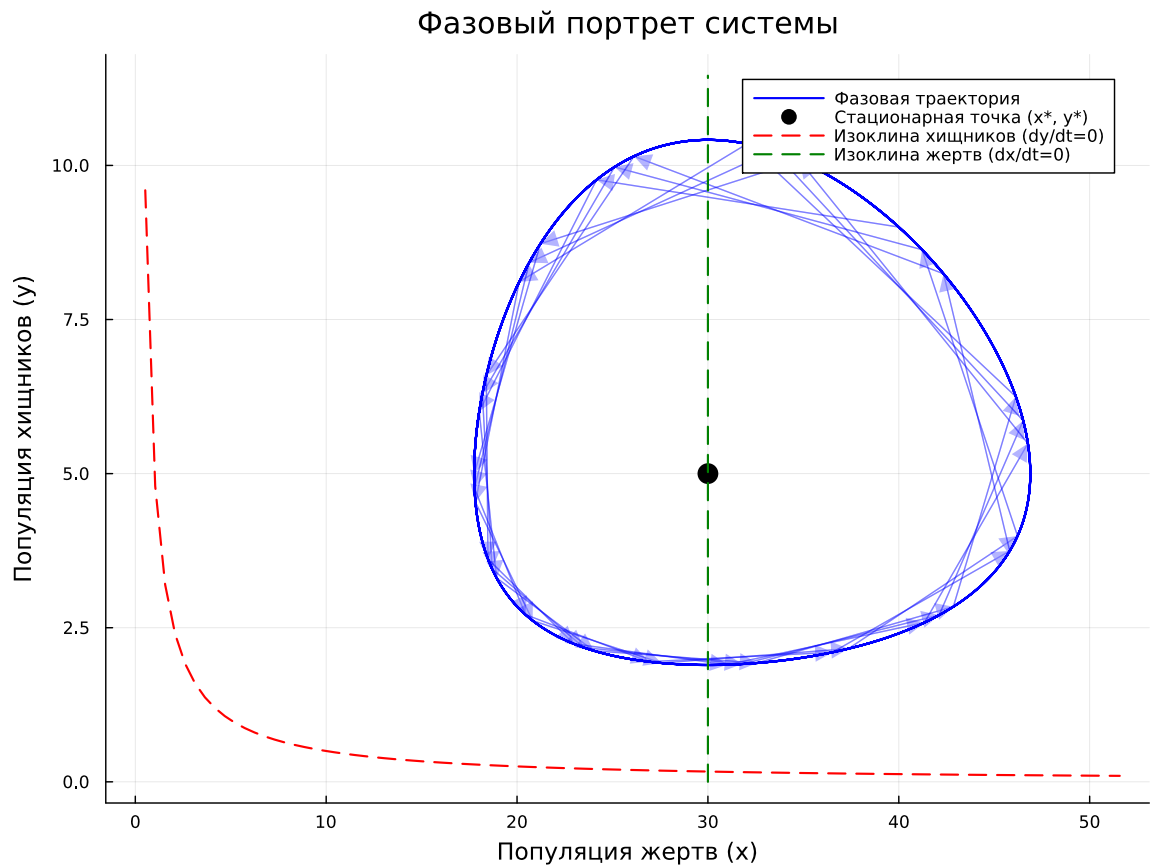
    x_range,
    y_nullcline,
    color = :red,
    linestyle = :dash,
    linewidth = 1.5,
    label = "Изоклина хищников (dy/dt=0)",
)

y_range =
    LinRange(0, maximum(df_lv.predator) * 1.1, 100)

x_nullcline =
    p_lv[4] ./
    (p_lv[3] .* ones(length(y_range)))

plot!(
    plt2,
    x_nullcline,
    y_range,
    color = :green,
    linestyle = :dash,
    linewidth = 1.5,
    label = "Изоклина жертв (dx/dt=0)",
)

```



4.14 Скорости изменения популяций

Рассмотрим производные dx/dt и dy/dt .

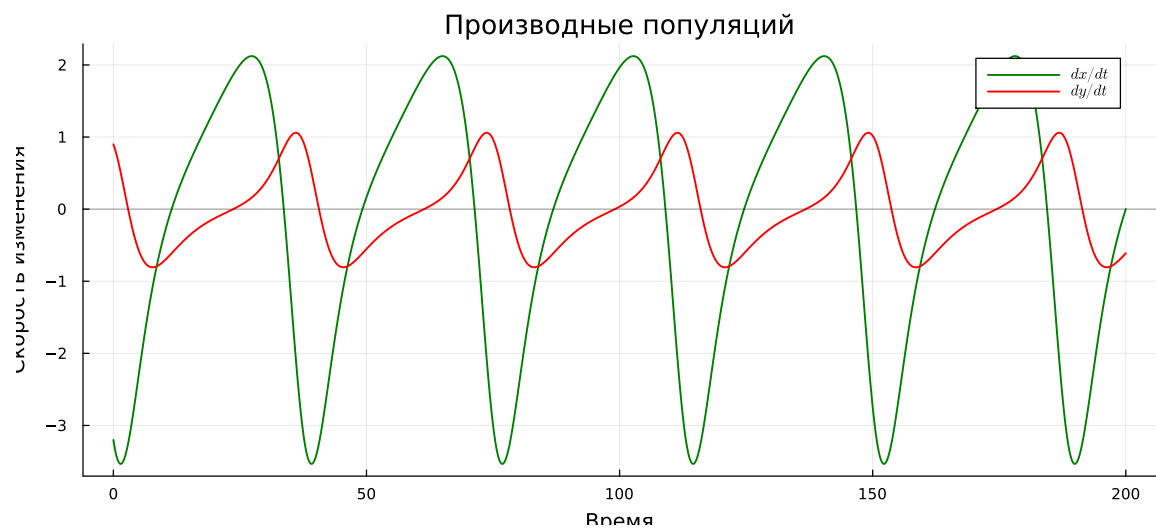
```
plt3 = plot(
    df_lv.t,
    [df_lv.dprey_dt df_lv.dpredator_dt],
    label = [L"dx/dt" L"dy/dt"],
    xlabel = "Время",
    ylabel = "Скорость изменения",
    title = "Производные популяций",
    linewidth = 1.5,
    legend = :topright,
```

```

    grid = true,
    size = (900, 400),
    color = [:green :red],
)

hline!(
    plt3,
    [0],
    color = :black,
    linestyle = :solid,
    alpha = 0.3,
    label = false,
)

```



4.15 Относительные темпы роста

Для сравнения динамики популяций вычислим относительные скорости изменения в процентах.

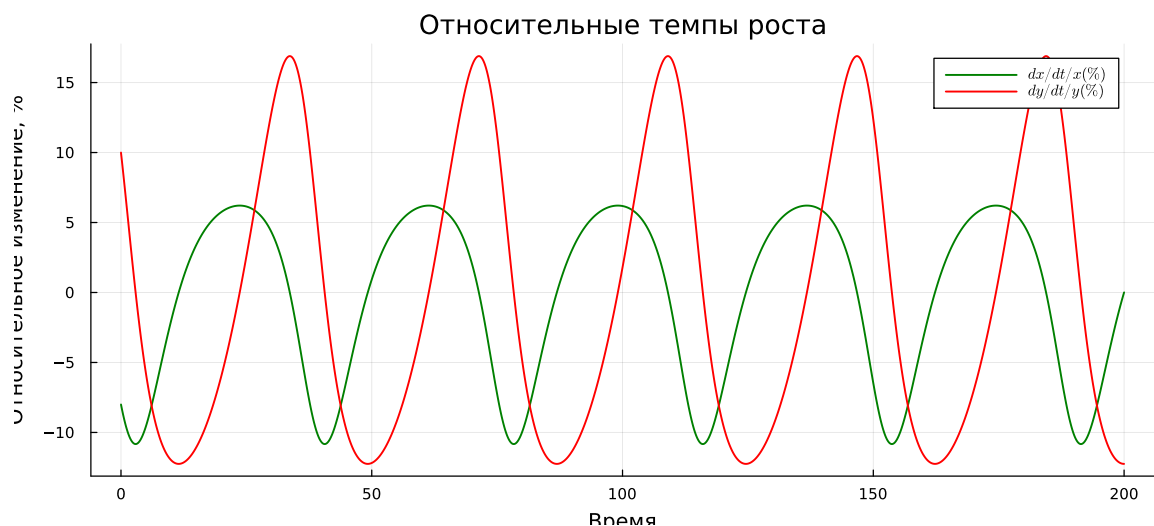
```

df_lv[!, :prey_pct_change] =
    df_lv.dprey_dt ./ df_lv.prey .* 100

df_lv[!, :predator_pct_change] =
    df_lv.dpredator_dt ./ df_lv.predator .* 100

plt4 = plot(
    df_lv.t,
    [df_lv.prey_pct_change df_lv.predator_pct_change],
    label = [L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel = "Время",
    ylabel = "Относительное изменение, %",
    title = "Относительные темпы роста",
    linewidth = 1.5,
    legend = :topright,
    grid = true,
    size = (900, 400),
    color = [:green :red],
)

```



4.16 Спектральный анализ

Поскольку решение модели имеет колебательный характер, выполним спектральный анализ с помощью быстрого преобразования Фурье.

```
function compute_fft(signal, dt)
    n = length(signal)
    spectrum = abs(rfft(signal))
    freq = rfftfreq(n, 1 / dt)
    return freq, spectrum
end
```

compute_fft (generic function with 1 method)

Перед FFT удаляем среднее значение сигнала, чтобы постоянная составляющая не доминировала в спектре.

```
dt_fft = 0.1

freq_pre, spectrum_pre =
    compute_fft(
        df_lv.prey .- mean(df_lv.prey),
        dt_fft,
    )

freq_pred, spectrum_pred =
    compute_fft(
        df_lv.predator .- mean(df_lv.predator),
        dt_fft,
    )
```

```

plt5 = plot(
    freq_prej,
    [spectrum_prej spectrum_predator],
    label = [L"Жертвы (x)" L"Хищники (y)"],
    xlabel = "Частота",
    ylabel = "Амплитуда",
    title = "Спектральный анализ (Фурье)",
    linewidth = 1.5,
    xscale = :log10,
    yscale = :log10,
    legend = :topright,
    grid = true,
    size = (800, 400),
    color = [:green :red],
)

```

```
└ Warning: No strict ticks found
```

```
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```

```
└ Warning: No strict ticks found
```

```
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```

```
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
```

```
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

```
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
```

```
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

```
└ Warning: No strict ticks found
```

```
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```

```
└ Warning: No strict ticks found
```

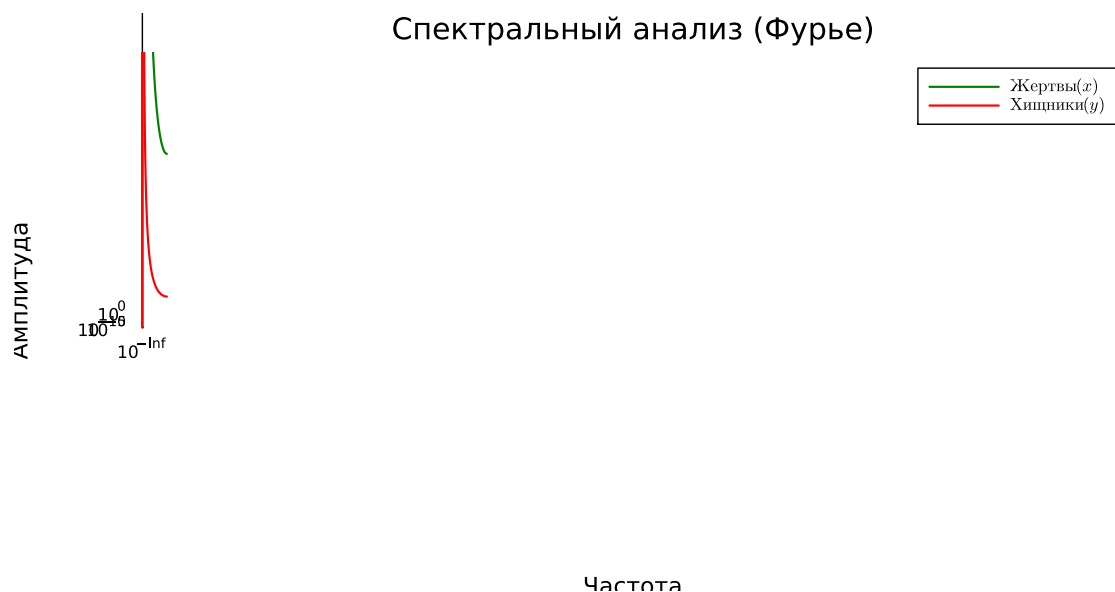
```
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```

```
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
```

```

└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194

```



```

└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale

```

```
L @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
L @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

4.16.1 Доминирующая частота

Найдём основной спектральный максимум для популяции жертв. Нулевая частота при поиске максимума исключается.

```
if length(spectrum_preym) > 0
    idx_preym =
        argmax(spectrum_preym[2:end]) + 1

    dominant_freq_preym =
        freq_preym[idx_preym]

    period_preym =
        1 / dominant_freq_preym

    println(
        "\nДоминирующая частота колебаний жертв: ",
        round(dominant_freq_preym, digits = 4),
        " Гц",
    )

    println(
        "Период колебаний жертв: ",
        round(period_preym, digits = 2),
        " единиц времени",
    )
end
```

```
end
```

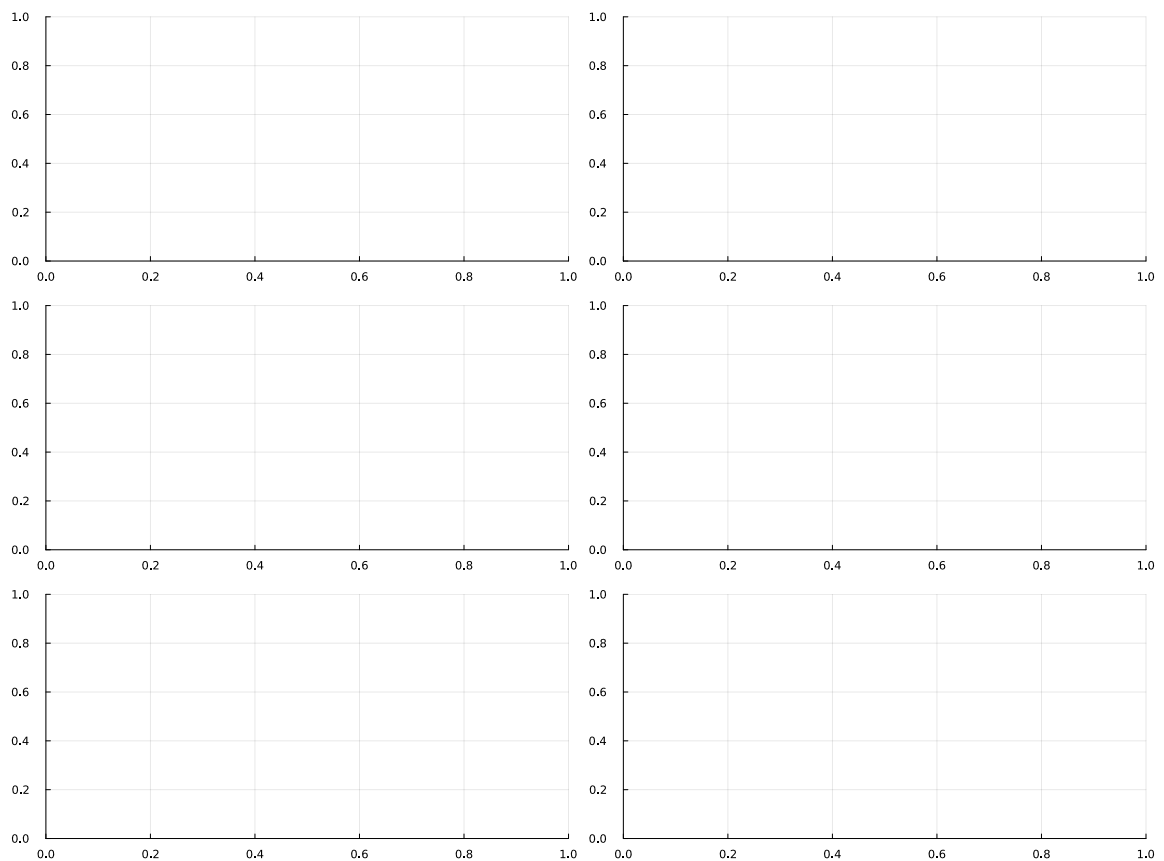
Доминирующая частота колебаний жертв: 0.025 Гц

Период колебаний жертв: 40.02 единиц времени

4.17 Сводная панель результатов

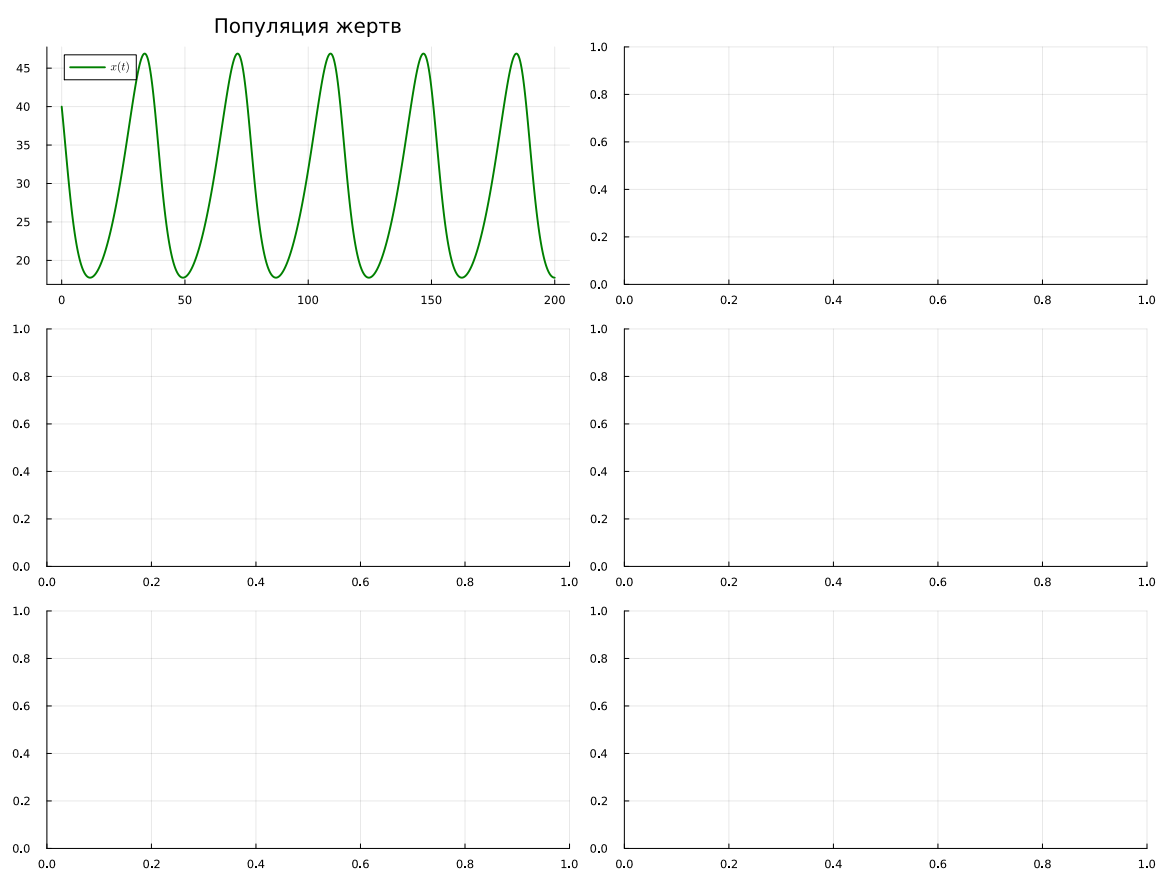
Разместим основные результаты анализа на одной панели.

```
plt6 = plot(  
    layout = (3, 2),  
    size = (1200, 900),  
)
```



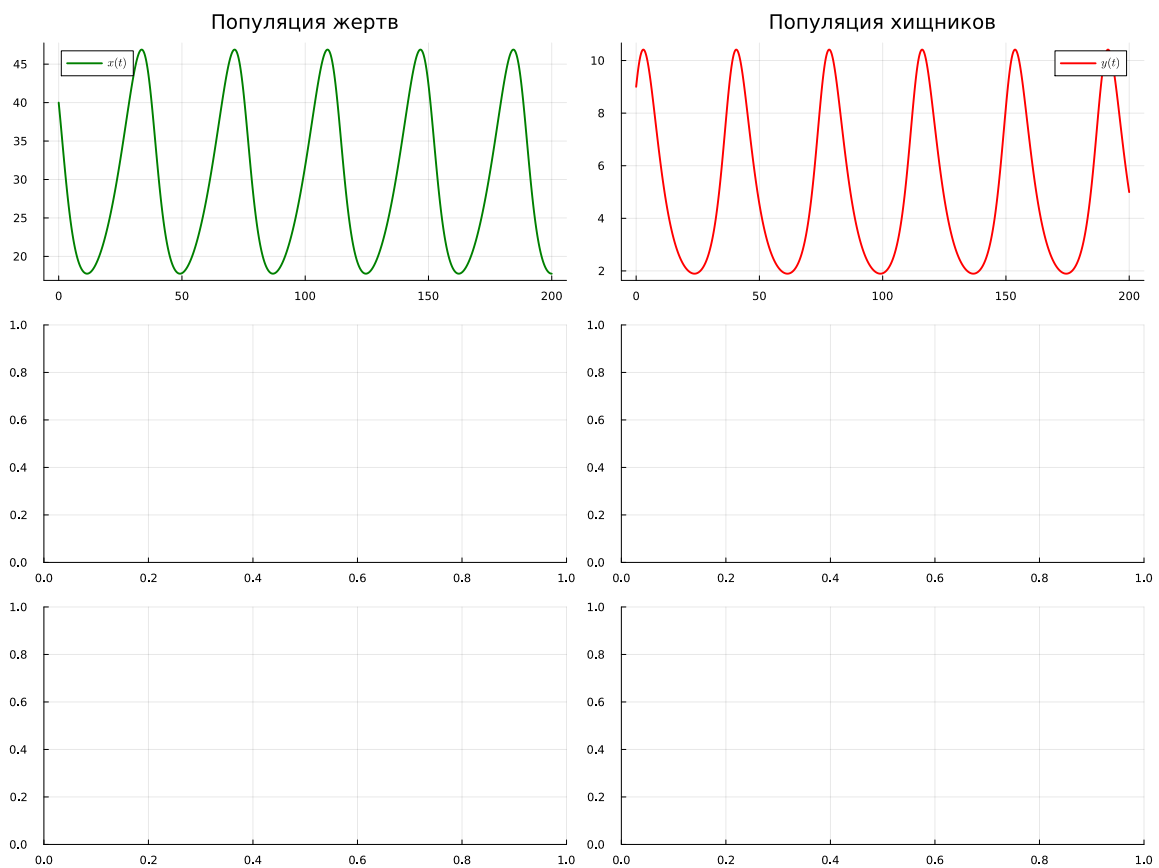
4.17.1 Динамика жертв

```
plot!(  
    plt6[1],  
    df_lv.t,  
    df_lv.prey,  
    label = L"x(t)",  
    color = :green,  
    linewidth = 2,  
    title = "Популяция жертв",  
    grid = true,  
)
```



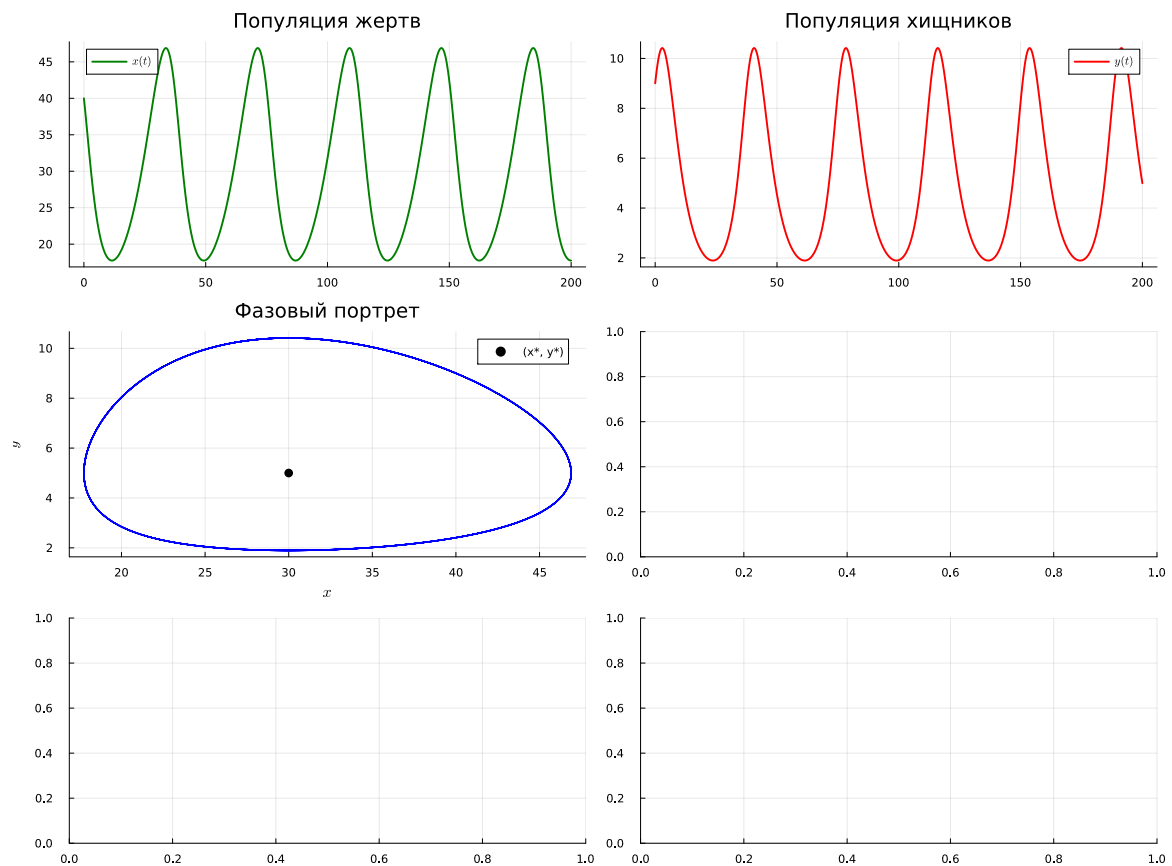
4.17.2 Динамика хищников

```
plot!(  
    plt6[2],  
    df_lv.t,  
    df_lv.predator,  
    label = L"y(t)",  
    color = :red,  
    linewidth = 2,  
    title = "Популяция хищников",  
    grid = true,  
)
```



4.17.3 Фазовый портрет

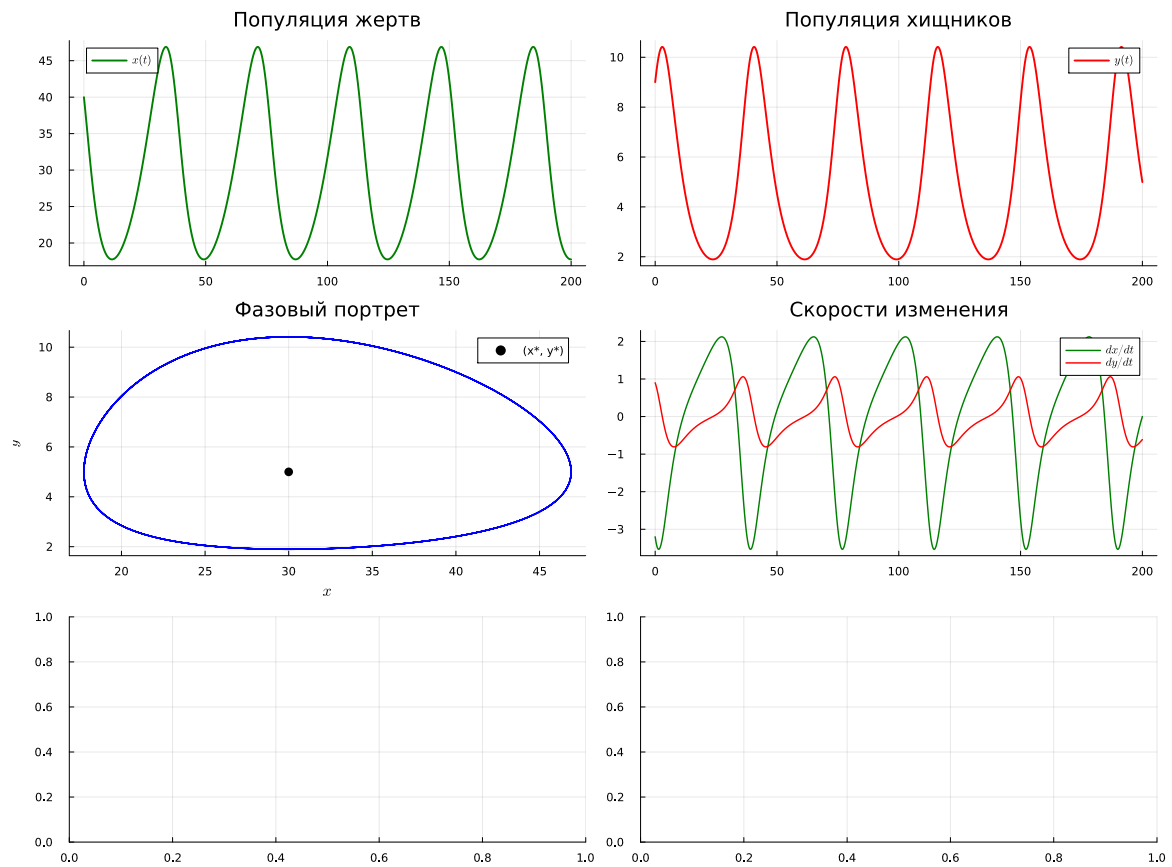
```
plot!(  
    plt6[3],  
    df_lv.prey,  
    df_lv.predator,  
    label = false,  
    color = :blue,  
    linewidth = 1.5,  
    title = "Фазовый портрет",  
    xlabel = L"x",  
    ylabel = L"y",  
    grid = true,  
)  
  
scatter!(  
    plt6[3],  
    [x_star],  
    [y_star],  
    color = :black,  
    markersize = 5,  
    label = "(x*, y*)",  
)
```



4.17.4 Производные

```
plot!(
    plt6[4],
    df_lv.t,
    [df_lv.dprey_dt df_lv.dpredator_dt],
    label = [L"dx/dt" L"dy/dt"],
    color = [:green :red],
    linewidth = 1.5,
    title = "Скорости изменения",
    grid = true,
    legend = :topright,
```

)

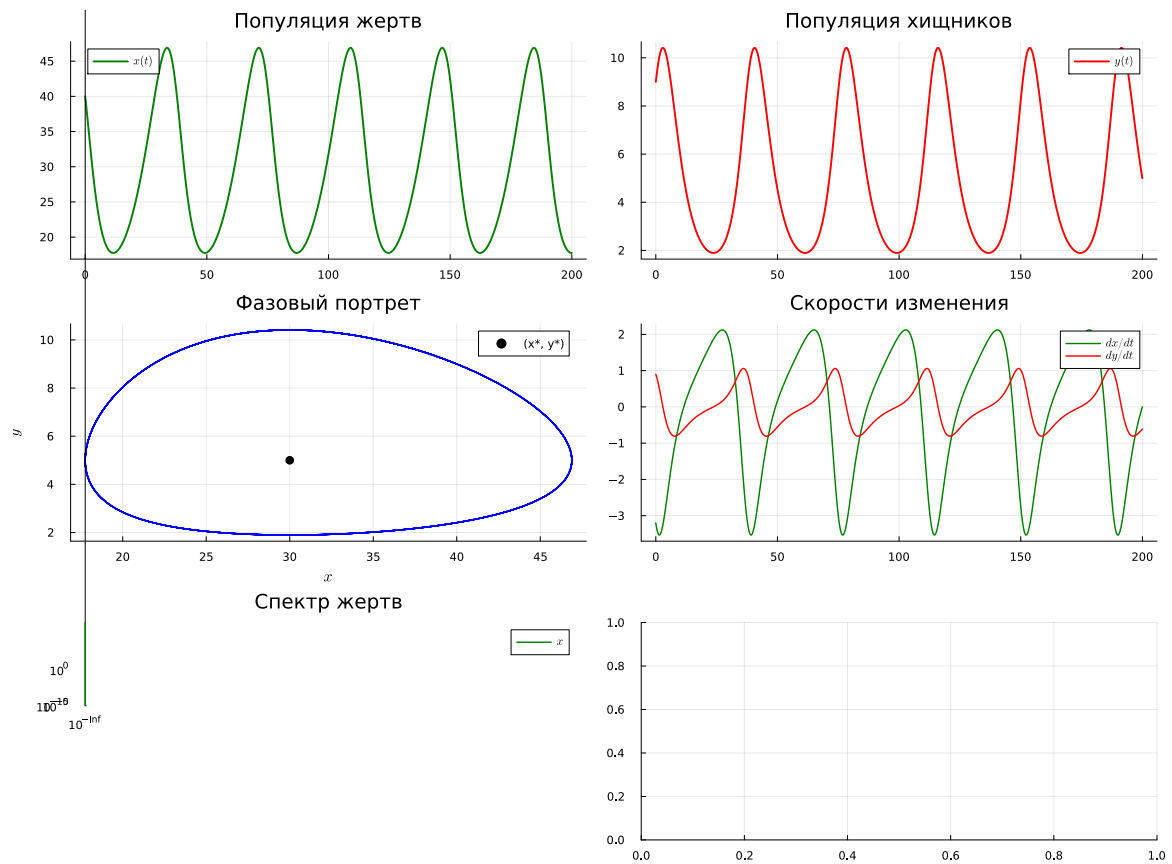


4.17.5 Спектр жертв

```
plot!(
    plt6[5],
    freq_prey,
    spectrum_prey,
    label = L"x",
    color = :green,
    linewidth = 1.5,
    title = "Спектр жертв",
    xscale = :log10,
```

```
yscale = :log10,  
grid = true,  
)
```

```
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```



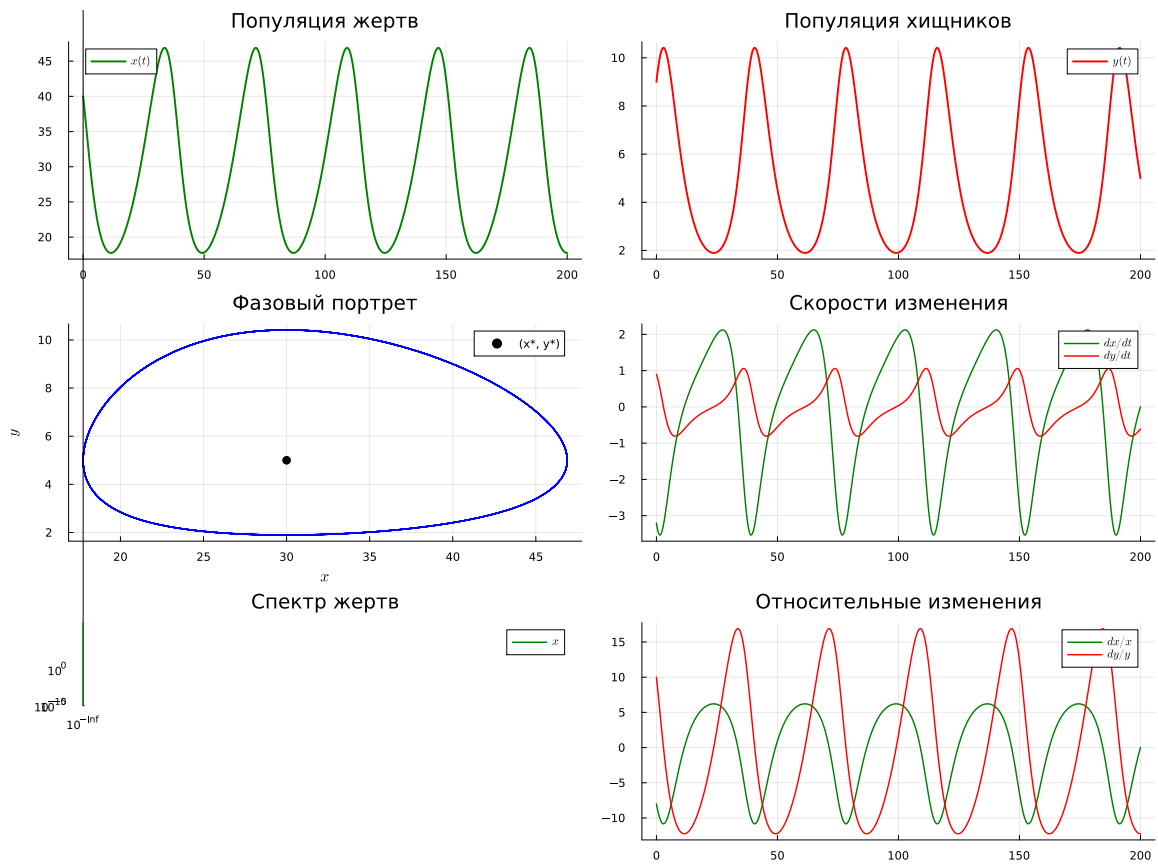
```
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

4.17.6 Относительные изменения

```
plot!(
    plt6[6],
    df_lv.t,
    [df_lv.prey_pct_change df_lv.predator_pct_change],
    label = [L"dx/x" L"dy/y"],
    color = [:green :red],
    linewidth = 1.5,
    title = "Относительные изменения",
```

```
    grid = true,  
    legend = :topright,  
)
```

```
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale  
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194  
└ Warning: No strict ticks found  
└ @ PlotUtils ~/.julia/packages/PlotUtils/J9gzB/src/ticks.jl:194
```



```
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

4.18 Анализ результатов

Выведем основные статистические характеристики обеих популяций.

```
println("\n" * "="^60)
println("Анализ результатов")
println("="^60)

println("\nОсновные статистики:")
```

```
println(
    "Жертвы: min = ",
    round(minimum(df_lv.prey), digits = 2),
    ", max = ",
    round(maximum(df_lv.prey), digits = 2),
    ", mean = ",
    round(mean(df_lv.prey), digits = 2),
)

println(
    "Хищники: min = ",
    round(minimum(df_lv.predator), digits = 2),
    ", max = ",
    round(maximum(df_lv.predator), digits = 2),
    ", mean = ",
    round(mean(df_lv.predator), digits = 2),
)
```

=====

Анализ результатов

=====

Основные статистики:

Жертвы: min = 17.75, max = 46.89, mean = 29.71

Хищники: min = 1.9, max = 10.41, mean = 5.2

4.19 Дополнительный анализ колебаний

Далее в исходном скрипте выполняется упрощённый анализ периодических колебаний без поиска локальных максимумов.

Продолжение исходного кода в присланном фрагменте отсутствует.

5 Выводы

В ходе лабораторной работы были реализованы и исследованы модели SIR и Лотки–Вольтерры. Для модели SIR была проанализирована динамика распространения инфекции и изменение эффективного репродуктивного числа. Для модели Лотки–Вольтерры исследованы колебания численности популяций жертв и хищников, определены стационарные точки и основные характеристики колебаний. Результаты моделирования были представлены графически и проанализированы в Jupyter Notebook.